



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №1
«Разведочный анализ данных.
Исследование и визуализация данных»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026г.

Цель лабораторной работы

Изучение различных методов визуализация данных.

Задание

1. Выбрать набор данных (датасет).
2. Для первой лабораторной работы рекомендуется использовать датасет без пропусков в данных.
3. Создать ноутбук, который содержит следующие разделы:
4. Текстовое описание выбранного Вами набора данных.
5. Основные характеристики датасета.
6. Визуальное исследование датасета.
7. Информация о корреляции признаков.
8. Сформировать отчет и разместить его в своем репозитории на github.

Текст программы и результаты

1) Текстовое описание набора данных

В качестве набора данных мы будем использовать датасет с информацией о пищевой ценности блюд из меню ресторанов быстрого питания Subway - <https://www.kaggle.com/datasets/joebeachcapital/subway-nutrition>

Эта задача является актуальной для анализа продуктов питания, помощи потребителям в выборе блюд в соответствии с их диетическими потребностями, а также для сравнения питательной ценности различных позиций меню.

Датасет представляет собой единый файл в формате CSV (или Excel), содержащий подробную информацию о различных продуктах, предлагаемых в Subway.

Файл содержит следующие колонки (на основе описания датасета):

- Category - Категория блюда.
- Item - Конкретный ингредиент.
- Calories - Калорийность блюда в килокалориях (ккал) - **целевой признак**.
- Total Fat (g) - Общее содержание жиров в граммах.
- Saturated Fat (g) - Содержание насыщенных жиров в граммах.
- Trans Fat (g) - Содержание трансжиров в граммах.
- Cholesterol (mg) - Содержание холестерина в миллиграммах.
- Sodium (mg) - Содержание натрия (соли) в миллиграммах.
- Carbs (g) - Общее содержание углеводов в граммах.
- Dietary Fiber (g) - Содержание пищевых волокон (клетчатки) в граммах.
- Sugars (g) - Содержание сахара в граммах.
- Protein (g) - Содержание белка в граммах.
- Weight Watchers Points - Баллы по системе Weight Watchers (система подсчета очков для контроля веса).

2) Импорт библиотек

Импортируем библиотеки с помощью команды `import`. Как правило, все команды `import` размещают в первой ячейке ноутбука, но мы в этом примере будем подключать все библиотеки последовательно, по мере их использования.

```
[2] import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set(style="ticks")
```

Python

3) Загрузка данных

```
[4] data = pd.read_csv('SubwayMenuNutrition.csv', sep=",")
```

Python

4) Основные характеристики датасета

```
[5] # Первые 5 строк датасета
data.head()
```

Python

...

	Category	Item	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Carbs (g)	Dietary Fiber (g)	Sugars (g)	Protein (g)	Weight Watchers Pnts
0	Sandwiches	BBQ Chicken	330	4.0	1.0	0.0	50	860	50	2	16	24	323.0
1	Sandwiches	BBQ Rib	580	31.0	10.0	0.0	60	1260	54	3	18	21	587.0
2	Sandwiches	Black Forest Ham	260	4.0	1.5	0.0	30	720	42	5	8	18	251.5
3	Sandwiches	Buffalo Chicken	360	12.0	2.0	0.0	55	7710	39	3	5	24	343.0
4	Sandwiches	Chicken & Bacon Ranch	530	26.0	10.0	0.5	100	1100	41	3	6	36	510.0

```
[7] # Размер датасета - 186 строк, 13 колонок
data.shape
```

Python

... (186, 13)

```
[7] # Размер датасета - 186 строк, 13 колонок
data.shape

Python
... (186, 13)
```

```
[8] total_count = data.shape[0]
print('Всего строк: {}'.format(total_count))

Python
... Всего строк: 186
```

```
[9] # Список колонок
data.columns

Python
... Index(['Category', 'Item', 'Calories', 'Total Fat (g)', 'Saturated Fat (g)',
        'Trans Fat (g)', 'Cholesterol (mg)', 'Sodium (mg)', 'Carbs (g)',
        'Dietary Fiber (g)', 'Sugars (g)', 'Protein (g)',
        'Weight Watchers Pnts'],
        dtype='object')
```

```
[10] # Список колонок с типами данных
data.dtypes

Python
... Category          object
Item                 object
Calories             int64
Total Fat (g)        float64
Saturated Fat (g)    float64
Trans Fat (g)        float64
Cholesterol (mg)      int64
Sodium (mg)          int64
Carbs (g)            int64
Dietary Fiber (g)     int64
Sugars (g)           int64
Protein (g)          int64
Weight Watchers Pnts float64
dtype: object
```

```
[11] # Проверим наличие пустых значений
# Цикл по колонкам датасета
for col in data.columns:
    # Количество пустых значений - все значения заполнены
    temp_null_count = data[data[col].isnull()].shape[0]
    print('{} - {}'.format(col, temp_null_count))

Python
... Category - 0
Item - 0
Calories - 0
Total Fat (g) - 0
Saturated Fat (g) - 0
Trans Fat (g) - 0
Cholesterol (mg) - 0
Sodium (mg) - 0
Carbs (g) - 0
Dietary Fiber (g) - 0
Sugars (g) - 0
Protein (g) - 0
Weight Watchers Pnts - 1
```

```
[12] # Основные статистические характеристики набора данных
data.describe()

Python

...

```

	Calories	Total Fat (g)	Saturated Fat (g)	Trans Fat (g)	Cholesterol (mg)	Sodium (mg)	Carbs (g)	Dietary Fiber (g)	Sugars (g)	Protein (g)	Weight Watchers Pnts
count	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000	185.000000
mean	316.612903	14.494624	4.876344	0.155914	51.424731	1028.037634	28.596774	2.844086	6.080645	19.215054	309.967568
std	235.703847	14.284559	5.175125	0.378321	48.390162	1419.398824	23.340113	2.427855	7.727029	15.179182	229.145913
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	122.500000	3.500000	1.000000	0.000000	5.000000	242.500000	10.000000	0.250000	2.250000	5.000000	121.000000
50%	290.000000	9.000000	2.500000	0.000000	45.000000	800.000000	31.500000	3.000000	5.000000	19.000000	286.500000
75%	470.000000	24.000000	8.000000	0.000000	90.000000	1427.500000	43.000000	4.000000	7.000000	30.000000	455.000000
max	1080.000000	65.000000	23.000000	2.000000	195.000000	14860.000000	114.000000	16.000000	65.000000	59.000000	1097.000000

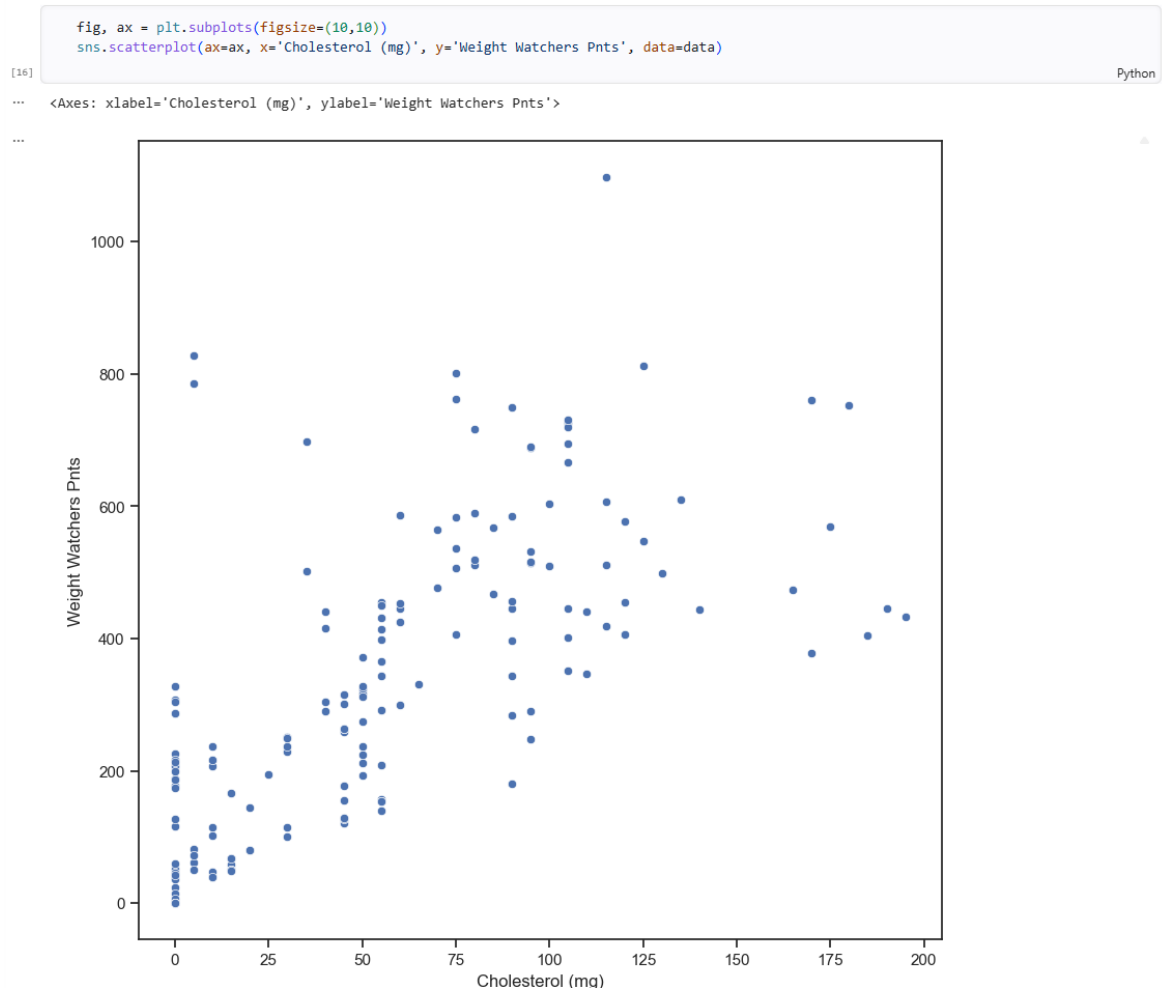
```
[14] # Определим уникальные значения для целевого признака
data['Calories'].unique()

Python

... array([ 330,  580,  260,  360,  530,  290,  340,  310,  430,  520,  380,
         420,  150,  230,  170,  200,  550,  490,  600,  270,  570,  280,
         450,  320,  470,  410,  780,  250,  720,  730,  190, 1080,  880,
         610,  710,  830,  680,  770,  790,  640,  820,  500,  630,  700,
         540,  750,  440,  760,  510,  590,  370,  300,  160,  120,  220,
         460,  240,  130,  140,  480,  210,  110,   50,  390,  180,   40,
          70,   0,   80,   60,   20,   45,    5,  100,   15,   35,   30,
          10])
```

5) Визуальное исследование датасета

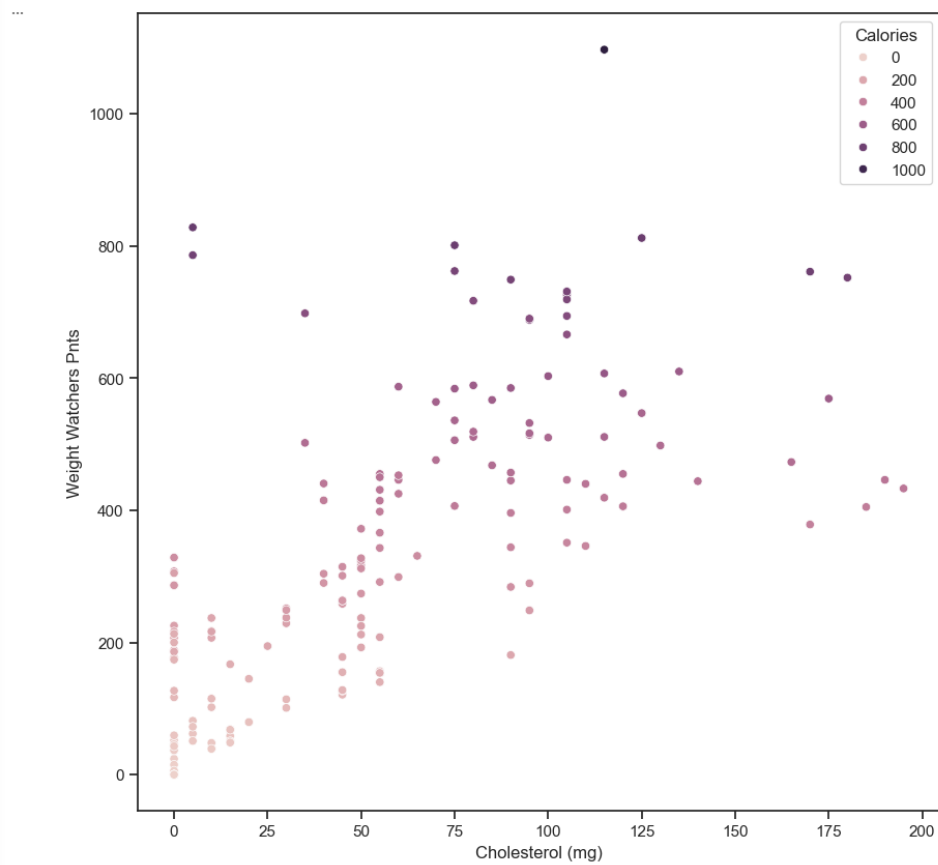
Диаграмма рассеяния



```
fig, ax = plt.subplots(figsize=(10,10))
sns.scatterplot(ax=ax, x='Cholesterol (mg)', y='Weight Watchers Pnts', data=data, hue='Calories')
```

[17] Python

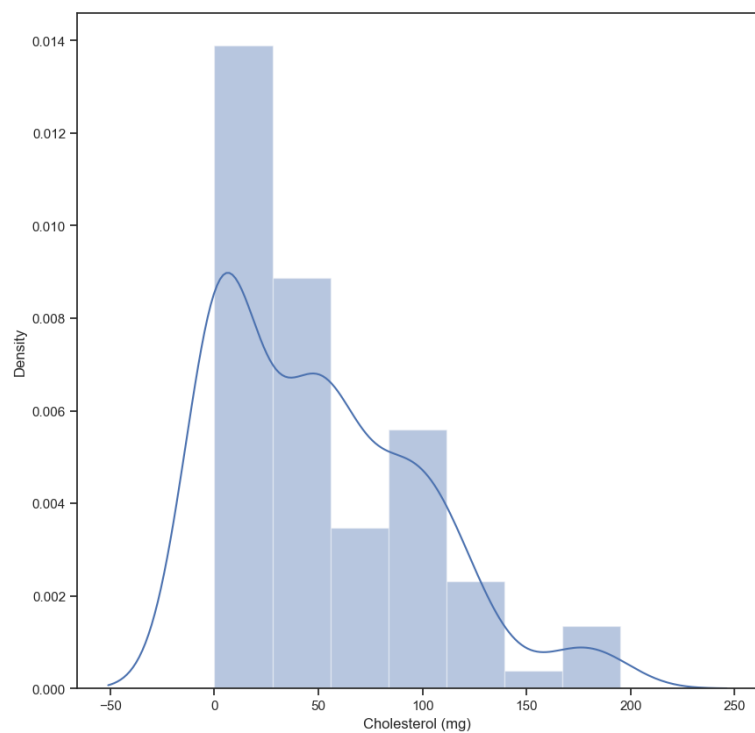
```
<Axes: xlabel='Cholesterol (mg)', ylabel='Weight Watchers Pnts'>
```



Гистограмма

```
fig, ax = plt.subplots(figsize=(10,10))
sns.distplot(data['Cholesterol (mg)'])
```

Python



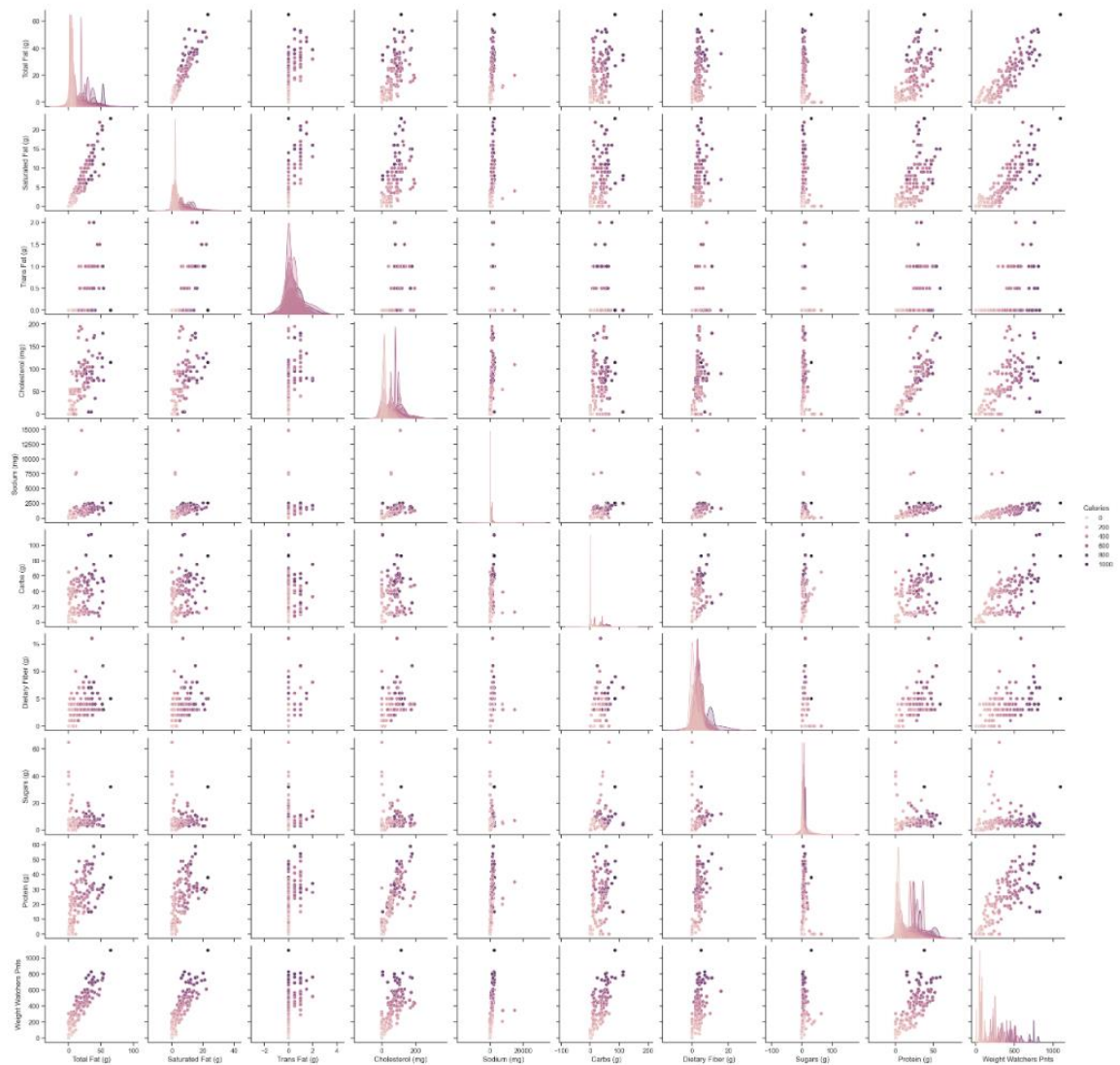
Парные диаграммы

```
sns.pairplot(data, hue="Calories")
```

[21]

Python

```
<seaborn.axisgrid.PairGrid at 0x200a3d9a4d0>
```



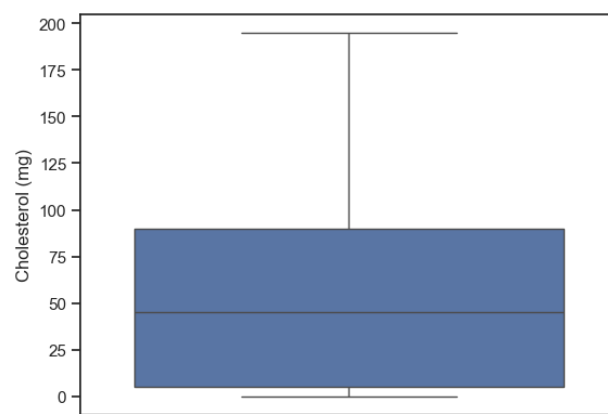
Ящик с усами

```
sns.boxplot(y=data['Cholesterol (mg)'])
```

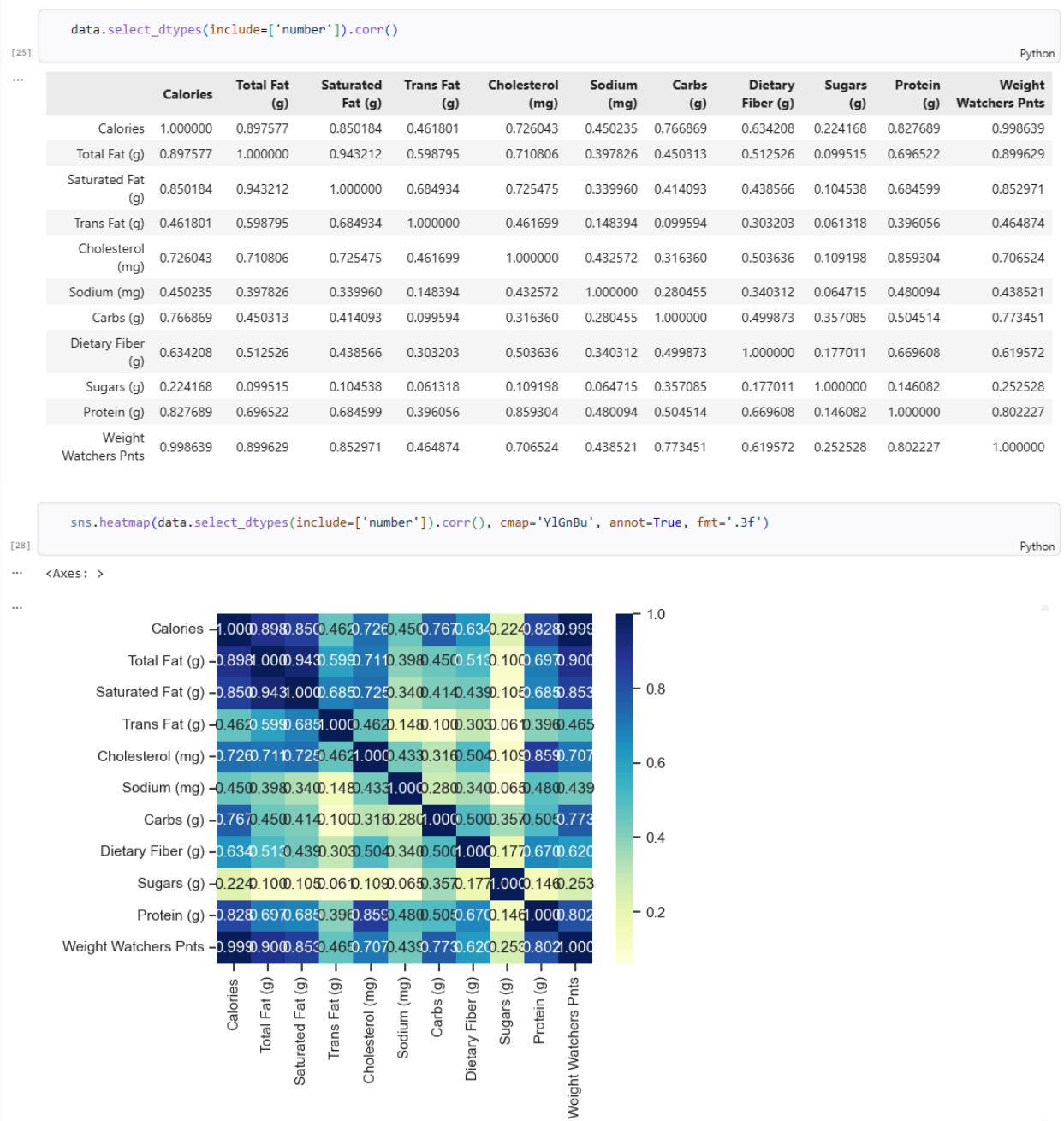
[23]

Python

```
<Axes: ylabel='Cholesterol (mg) '>
```



6) Информация о корреляции признаков



Ключевые наблюдения:

- Гистограмма Cholesterol (mg) имеет сильную правостороннюю асимметрию;
- Большинство блюд (более 100) имеют низкий уровень холестерина (от 0 до ~50 мг);
- Присутствуют значительные выбросы в сторону высоких значений (один из образцов достигает почти 200 мг, а максимум, судя по data.describe(), составляет 195 мг, но на гистограмме виден пик около 7700 — возможно, это

опечатка в данных или какой-то уникальный продукт);

- Парные диаграммы (pairplot), раскрашенные по калориям, показывают, что блюда с высокой калорийностью (светлые точки на вашем графике) обычно имеют и высокое содержание жиров, углеводов и баллов WW. Это формирует отдельный кластер в правом верхнем углу графиков;

- Блюда с низкой калорийностью (темные точки, вероятно, салаты и легкие закуски) образуют плотную группу в нижнем левом углу;

- Жиры и калории: очевидна прямая зависимость: чем выше Total Fat, тем выше Calories;

- Белки и калории: корреляция также положительная, но менее тесная, чем с жирами. Некоторые продукты могут быть богаты белком, но иметь умеренную калорийность (например, куриные закуски без соуса);

- Углеводы и сахар: на парных графиках видна группа блюд с высоким содержанием углеводов, но низким содержанием сахара (вероятно, цельнозерновой хлеб или несладкие закуски), и другая группа, где углеводы сопровождаются высоким сахаром (сладкие соусы, печенье);

- Некоторые признаки, например Dietary Fiber, слабо коррелируют с калориями и жирами. Много клетчатки не означает автоматически высокую калорийность (салаты, овощи), и наоборот (жирные бургеры могут содержать мало клетчатки).

Выводы по работе

Проведен полный EDA: данные чистые, сбалансированные, не требуют импутации. Выявлена высокая корреляция между размерами лепестков, что указывает на их высокую информативность для классификации. Данные готовы к обучению моделей.[github](#).



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №2
«Обработка пропусков в данных, кодирование категориальных
признаков, масштабирование данных»
по дисциплине «Технологии машинного обучения»**

**Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата**

**Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата**

2026г.

Цель лабораторной работы

Изучение способов предварительной обработки данных для дальнейшего формирования моделей.

Задание

1. Выбрать набор данных (датасет), содержащий категориальные признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.)

2. Для выбранного датасета (датасетов) на основе материалов лекции решить следующие задачи:

- обработку пропусков в данных;
- кодирование категориальных признаков;
- масштабирование данных.

Текст программы и результаты

В данной работе исследуется набор данных Airplane Crashes and Fatalities Since 1908, содержащий сведения об авиационных происшествиях за длительный исторический период.

- Цель: Подготовить данные для дальнейшего обучения моделей.
- Особенности: Датасет содержит пропуски в числовых и категориальных данных, а также признаки разных типов и шкал.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set(style="ticks")
```

Загрузка и первичный анализ данных

```
[2] data = pd.read_csv('Airplane_Crashes_and_Fatalities_Since_1908.csv', sep=",")
```

Python

```
[3] # размер набора данных
data.shape
```

Python

... (5268, 14)

```
[4] # типы колонок
data.dtypes
```

Python

```
... index          int64
Date              object
Time              object
Location          object
Operator          object
Flight #          object
Route             object
Type              object
Registration      object
cn/In             object
Aboard            float64
Fatalities        float64
Ground            float64
Summary           object
dtype: object
```

```
[5] # проверим есть ли пропущенные значения
data.isnull().sum()
```

Python

```
... index          0
Date              0
Time              2219
Location          20
Operator          18
Flight #          4199
Route             1707
Type              27
Registration      335
cn/In             1228
Aboard            22
Fatalities        12
Ground            22
Summary           390
dtype: int64
```

```
[6] # Первые 5 строк датасета
data.head()
```

Python

	index	Date	Time	Location	Operator	Flight #	Route	Type	Registration	cn/In	Aboard	Fatalities	Ground	Summary
0	0	09/17/1908	17:18	Fort Myer, Virginia	Military - U.S. Army	NaN	Demonstration	Wright Flyer III	NaN	1	2.0	1.0	0.0	During a demonstration flight, a U.S. Army fly...
1	1	07/12/1912	06:30	AtlantiCity, New Jersey	Military - U.S. Navy	NaN	Test flight	Dirigible	NaN	NaN	5.0	5.0	0.0	First U.S. dirigible Akron exploded just offsh...
2	2	08/06/1913	NaN	Victoria, British Columbia, Canada	Private	-	NaN	Curtiss seaplane	NaN	NaN	1.0	1.0	0.0	The first fatal airplane accident in Canada oc...
3	3	09/09/1913	18:30	Over the North Sea	Military - German Navy	NaN	NaN	Zeppelin L-1 (airship)	NaN	NaN	20.0	14.0	0.0	The airship flew into a thunderstorm and encou...
4	4	10/17/1913	10:30	Near Johannisthal, Germany	Military - German Navy	NaN	NaN	Zeppelin L-2 (airship)	NaN	NaN	30.0	30.0	0.0	Hydrogen gas which was being vented was sucked...

```
[7] total_count = data.shape[0]
print('Всего строк: {}'.format(total_count))
```

Python

... Всего строк: 5268

Обработка пропусков в данных

Мы применим два подхода:

1. Числовые признаки: заполним нулями;
2. Категориальные признаки: заполним самым частым значением (модой).

1) Обработка пропусков в данных

```
# Выберем числовые колонки с пропущенными значениями
# Цикл по колонкам датасета
num_cols = []
for col in data.columns:
    # Количество пустых значений
    temp_null_count = data[data[col].isnull()].shape[0]
    dt = str(data[col].dtype)
    if temp_null_count > 0 and (dt == 'float64' or dt == 'int64'):
        num_cols.append(col)
        temp_perc = round((temp_null_count / total_count) * 100.0, 2)
        print('Колонка {}. Тип данных {}. Количество пустых значений {},
        {}.format(col, dt, temp_null_count, temp_perc))
```

```
... Колонка Aboard. Тип данных float64. Количество пустых значений 22, 0.42%.
Колонка Fatalities. Тип данных float64. Количество пустых значений 12, 0.23%.
Колонка Ground. Тип данных float64. Количество пустых значений 22, 0.42%.
```

```
# Фильтр по колонкам с пропущенными значениями
```

```
data_num = data[num_cols]
```

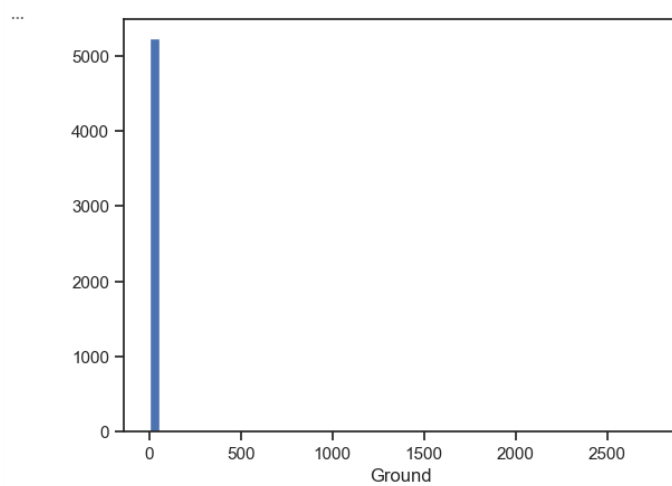
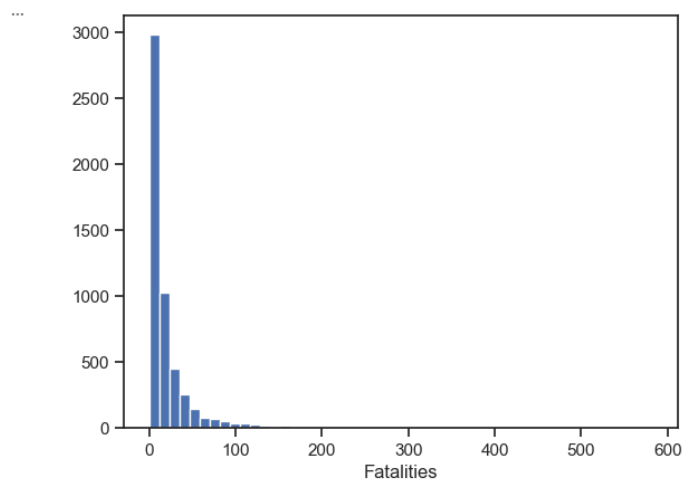
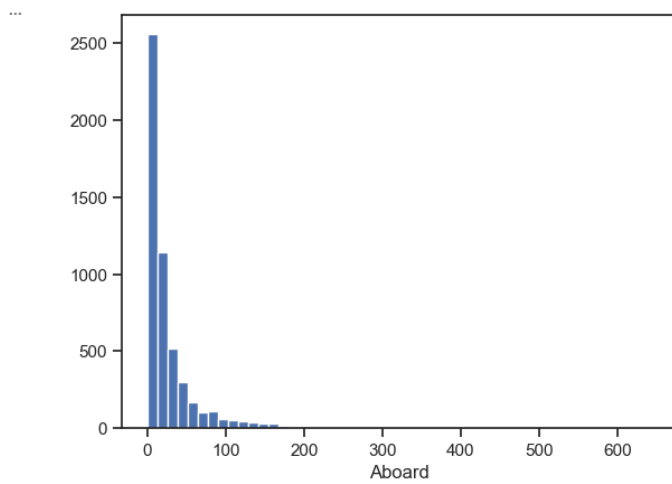
```
data_num
```

```
...
  Aboard  Fatalities  Ground
0      2.0         1.0     0.0
1      5.0         5.0     0.0
2      1.0         1.0     0.0
3     20.0        14.0     0.0
4     30.0        30.0     0.0
...      ...         ...     ...
5263   112.0        98.0     2.0
5264     4.0         4.0    NaN
5265   228.0       228.0     0.0
5266     1.0         1.0     0.0
5267    13.0        13.0     0.0
```

```
5268 rows × 3 columns
```

```
# Гистограмма по признакам
```

```
for col in data_num:
    plt.hist(data[col], 50)
    plt.xlabel(col)
    plt.show()
```



```
# Заполнение нулями только численных значений
data_new_3 = data.copy()
numeric_cols = data_new_3.select_dtypes(include=['int64', 'float64']).columns
data_new_3[numeric_cols] = data_new_3[numeric_cols].fillna(0)
data_new_3.head()
```

...

	index	Date	Time	Location	Operator	Flight #	Route	Type	Registration	cn/In	Aboard	Fatalities	Ground	Summary	
0	0	09/17/1908	17:18	Fort Myer, Virginia	Military - U.S. Army	NaN	Demonstration	Wright Flyer III		NaN	1	2.0	1.0	0.0	During a demonstration flight, a U.S. Army fly...
1	1	07/12/1912	06:30	AtlantiCity, New Jersey	Military - U.S. Navy	NaN	Test flight	Dirigible		NaN	NaN	5.0	5.0	0.0	First U.S. dirigible Akron exploded just offsh...
2	2	08/06/1913	NaN	Victoria, British Columbia, Canada	Private	-	NaN	Curtiss seaplane		NaN	NaN	1.0	1.0	0.0	The first fatal airplane accident in Canada oc...
3	3	09/09/1913	18:30	Over the North Sea	Military - German Navy	NaN	NaN	Zeppelin L-1 (airship)		NaN	NaN	20.0	14.0	0.0	The airship flew into a thunderstorm and encou...
4	4	10/17/1913	10:30	Near Johannisthal, Germany	Military - German Navy	NaN	NaN	Zeppelin L-2 (airship)		NaN	NaN	30.0	30.0	0.0	Hydrogen gas which was being vented was sucked...

4

5

Выберем категориальные колонки с пропущенными значениями

Цикл по колонкам датасета

```
cat_cols = []
```

```
for col in data.columns:
```

```
    # Количество пустых значений
```

```
    temp_null_count = data[data[col].isnull()].shape[0]
```

```
    dt = str(data[col].dtype)
```

```
    if temp_null_count>0 and (dt=='object'):
```

```
        cat_cols.append(col)
```

```
        temp_perc = round((temp_null_count / total_count) * 100.0, 2)
```

```
        print('Колонка {}. Тип данных {}. Количество пустых значений {},
```

```
{}}%.'.format(col, dt, temp_null_count, temp_perc))
```

... Колонка Time. Тип данных object. Количество пустых значений 2219, 42.12%.

Колонка Location. Тип данных object. Количество пустых значений 20, 0.38%.

Колонка Operator. Тип данных object. Количество пустых значений 18, 0.34%.

Колонка Flight #. Тип данных object. Количество пустых значений 4199, 79.71%.

Колонка Route. Тип данных object. Количество пустых значений 1707, 32.4%.

Колонка Type. Тип данных object. Количество пустых значений 27, 0.51%.

Колонка Registration. Тип данных object. Количество пустых значений 335, 6.36%.

Колонка cn/In. Тип данных object. Количество пустых значений 1228, 23.31%.

Колонка Summary. Тип данных object. Количество пустых значений 390, 7.4%.

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.impute import MissingIndicator
```

```
cat_cols = ['Time', 'Location', 'Operator', 'Flight #', 'Route', 'Type', 'Regis-
tration', 'cn/In', 'Summary']
```

```
cat_temp_data = data[cat_cols].copy()
```

Импыютация наиболее частыми значениями

```
imp2 = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
```

```
data_imp2_array = imp2.fit_transform(cat_temp_data)
```

```
data_imp2_df = pd.DataFrame(data_imp2_array, columns=cat_cols)
```

```
print("\nПОСЛЕ импыютации:")
```

```
for col in cat_cols:
```

```
    print(f"{col}: {data_imp2_df[col].isnull().sum()} пропусков")
```

```

...
ПОСЛЕ импутации:
Time: 0 пропусков
Location: 0 пропусков
Operator: 0 пропусков
Flight #: 0 пропусков
Route: 0 пропусков
Type: 0 пропусков
Registration: 0 пропусков
cn/In: 0 пропусков
Summary: 0 пропусков

data_cleaned = data_new_3.copy()
cat_cols = ['Time', 'Location', 'Operator', 'Flight #', 'Route', 'Type', 'Registration', 'cn/In', 'Summary']

data_cleaned = data_cleaned.drop(columns=cat_cols)

data_cleaned = pd.concat([data_cleaned, data_imp2_df], axis=1)

print("\nПервые 5 строк после объединения:")
data_cleaned.head()

```

Первые 5 строк после объединения:

	index	Date	Aboard	Fatalities	Ground	Time	Location	Operator	Flight #	Route	Type	Registration	cn/In	Summary
0	0	09/17/1908	2.0	1.0	0.0	17:18	Fort Myer, Virginia	Military - U.S. Army	-	Demonstration	Wright Flyer III	49	1	During a demonstration flight, a U.S. Army fly...
1	1	07/12/1912	5.0	5.0	0.0	06:30	AtlantiCity, New Jersey	Military - U.S. Navy	-	Test flight	Dirigible	49	178	First U.S. dirigible Akron exploded just offsh...
2	2	08/06/1913	1.0	1.0	0.0	15:00	Victoria, British Columbia, Canada	Private	-	Training	Curtiss seaplane	49	178	The first fatal airplane accident in Canada oc...
3	3	09/09/1913	20.0	14.0	0.0	18:30	Over the North Sea	Military - German Navy	-	Training	Zeppelin L-1 (airship)	49	178	The airship flew into a thunderstorm and encou...
4	4	10/17/1913	30.0	30.0	0.0	10:30	Near Johannisthal, Germany	Military - German Navy	-	Training	Zeppelin L-2 (airship)	49	178	Hydrogen gas which was being vented was sucked...

Кодирование категориальных признаков

Для обучения моделей машинного обучения текстовые данные должны быть преобразованы в числа.

2) Кодирование категориальных признаков

```

cat_enc = pd.DataFrame({'c1': data_cleaned['Operator']})
cat_enc.head()

```

[18] Python

	c1
0	Military - U.S. Army
1	Military - U.S. Navy
2	Private
3	Military - German Navy
4	Military - German Navy

Label Encoding

Использование LabelEncoder

```
[19] from sklearn.preprocessing import LabelEncoder Python
```

```
[20] cat_enc['c1'].unique() Python
```

```
[21] array(['Military - U.S. Army', 'Military - U.S. Navy', 'Private', ...,
      'Aerotuy airline', 'Bako Air', 'Strait Air'],
      shape=(2476,), dtype=object)
```

```
[22] le = LabelEncoder()
cat_enc_le = le.fit_transform(cat_enc['c1']) Python
```

```
[23] # Наименования категорий в соответствии с порядковыми номерами
# Свойство называется classes, потому что предполагается что мы решаем
# задачу классификации и каждое значение категории соответствует
# какому-либо классу целевого признака
le.classes_ Python
```

```
[24] array(['A B Aerotransport', 'AB Aerotransport', 'ACES Colombia', ...,
      'Zantop International Airlines', 'Zen Nippon',
      'de Havilland Aircraft'], shape=(2476,), dtype=object)
```

```
[25] cat_enc_le Python
```

```
[26] array([1566, 1577, 1823, ..., 197, 2122, 1474], shape=(5268,))
```

```
[27] np.unique(cat_enc_le) Python
```

```
[28] array([ 0,  1,  2, ..., 2473, 2474, 2475], shape=(2476,))
```

```
[29] le.inverse_transform([0, 1, 2, 3]) Python
```

```
[30] array(['A B Aerotransport', 'AB Aerotransport', 'ACES Colombia',
      'ADC Airlines'], dtype=object)
```

One-Hot Encoding

Используется для признаков с несколькими категориями, чтобы не вводить ложный "порядок" между ним.

Кодирование категорий наборами бинарных значений - one-hot encoding

```
[26] from sklearn.preprocessing import OneHotEncoder Python
```

```
[27] ohe = OneHotEncoder()
cat_enc_ohe = ohe.fit_transform(cat_enc[['c1']]) Python
```

```
[28] cat_enc.shape Python
```

```
[29] (5268, 1)
```

```
[29] cat_enc_ohc.shape
Python
... (5268, 2476)

[30] cat_enc_ohc
Python
... <Compressed Sparse Row sparse matrix of dtype 'float64'
    with 5268 stored elements and shape (5268, 2476)>

[31] cat_enc_ohc.todense()[0:10]
Python
... matrix([[0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            ...,
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 0., 0., 0.]], shape=(10, 2476))

[32] cat_enc.head(10)
Python
...
   c1
0    Military - U.S. Army
1    Military - U.S. Navy
2         Private
3    Military - German Navy
4    Military - German Navy
5    Military - German Navy
6    Military - German Navy
7    Military - German Army
8    Military - German Navy
9    Military - German Navy
```

Масштабирование данных

Признаки «Aboard» и «Fatalities» имеют разные диапазоны. Применим два популярных метода.

Min-Max Scaling (Нормализация)

Приводит данные к диапазону [0, 1].

MinMax масштабирование

```
[36] from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer
Python

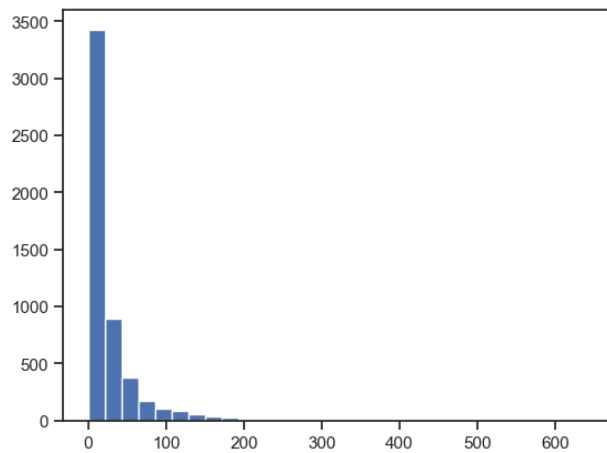
[37] sc1 = MinMaxScaler()
    sc1_data = sc1.fit_transform(data_cleaned[['Aboard']])
Python
```

```
plt.hist(data['Aboard'], 30)
plt.show()
```

[38]

Python

...

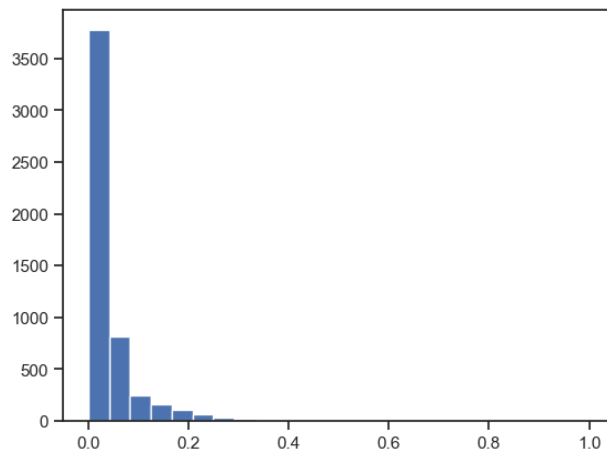


```
plt.hist(sc1_data, 24)
plt.show()
```

[39]

Python

...



StandardScaler (Стандартизация)

Приводит данные к среднему 0 и стандартному отклонению 1. Полезно для алгоритмов, чувствительных к распределению.

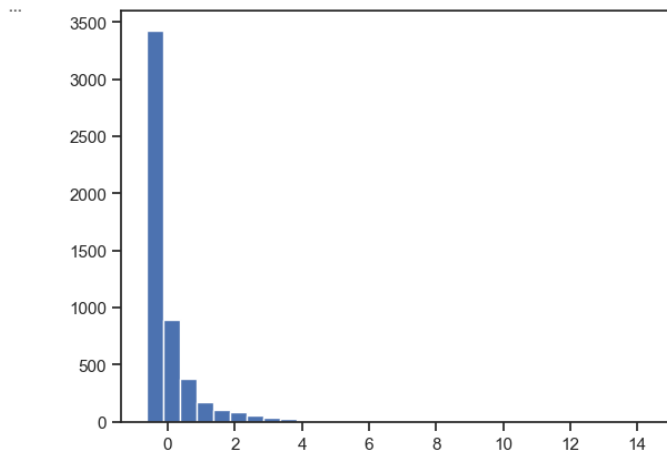
Масштабирование данных на основе Z-оценки - StandardScaler

```
[40] sc2 = StandardScaler()  
      sc2_data = sc2.fit_transform(data[['Aboard']])
```

Python

```
[41] plt.hist(sc2_data, 30)  
      plt.show()
```

Python



Краткие выводы по работе:

1. Пропуски: были заполнены статистическими методами (нули и мода), что позволило сохранить объем выборки;
2. Категории: были преобразованы в числовой вид. Для авиакомпаний использовано бинарное кодирование и One-Hot Encoding;
3. Масштабирование: признаки приведены к единому масштабу, что необходимо для корректной работы линейных моделей и метода ближайших соседей (k-NN).



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №3
«Подготовка обучающей и тестовой выборки, кросс-валидация и подбор
гиперпараметров на примере метода ближайших соседей»
по дисциплине «Технологии машинного обучения»**

**Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата**

**Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата**

2026г.

Цель лабораторной работы

Изучение способов предварительной обработки данных для дальнейшего формирования моделей.

Задание

1. Выбрать набор данных (датасет), содержащий категориальные признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.)

2. Для выбранного датасета (датасетов) на основе материалов лекции решить следующие задачи:

- обработку пропусков в данных;
- кодирование категориальных признаков;
- масштабирование данных.

Текст программы и результаты

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
%matplotlib inline
sns.set(style="ticks")
```

Загрузка и первичный анализ данных

```
[34] data = pd.read_csv('avocado.csv', sep=",") Python
```

```
[35] data.shape Python
```

```
... (18249, 14)
```

```
data.dtypes
```

```
[36] Python
```

```
... Unnamed: 0      int64
Date            object
AveragePrice    float64
Total Volume    float64
4046            float64
4225            float64
4770            float64
Total Bags     float64
Small Bags     float64
Large Bags     float64
XLarge Bags    float64
type           object
year          int64
region        object
dtype: object
```

```
data.isnull().sum()
```

```
[37] Python
```

```
... Unnamed: 0      0
Date            0
AveragePrice    0
Total Volume    0
4046            0
4225            0
4770            0
Total Bags     0
Small Bags     0
Large Bags     0
XLarge Bags    0
type           0
year           0
region         0
dtype: int64
```

```
data.head()
```

```
[38] Python
```

```
...
   Unnamed: 0  Date      AveragePrice  Total Volume  4046  4225  4770  Total Bags  Small Bags  Large Bags  XLarge Bags  type  year  region
0           0  2015-12-27           1.33    64236.62  1036.74  54454.85  48.16   8696.87   8603.62    93.25    0.0  conventional  2015  Albany
1           1  2015-12-20           1.35    54876.98   674.28  44638.81  58.33   9505.56   9408.07    97.49    0.0  conventional  2015  Albany
2           2  2015-12-13           0.93   118220.22   794.70  109149.67  130.50  8145.35   8042.21   103.14    0.0  conventional  2015  Albany
3           3  2015-12-06           1.08    78992.15  1132.00  71976.41  72.58   5811.16   5677.40   133.76    0.0  conventional  2015  Albany
4           4  2015-11-29           1.28    51039.60   941.48  43838.39  75.78   6183.95   5986.26   197.69    0.0  conventional  2015  Albany
```

```
total_count = data.shape[0]
print('Всего строк: {}'.format(total_count))
```

```
[39] Python
```

```
... Всего строк: 18249
```

Кодирование категориальных признаков

```
[40] data_new = data.drop(['Date'], axis=1) Python
```

```
[41] data_new = pd.get_dummies(data, columns=['region'], drop_first=True) Python
```

```
[42] data_new['type'] = data['type'].map({'conventional': 0, 'organic': 1}) Python
```

```
[43] data_new.head() Python
```

```
...
```

	Unnamed: 0	Date	AveragePrice	Total Volume	4046	4225	4770	Total Bags	Small Bags	Large Bags	...	region_SouthCarolina	region_SouthCentral
0	0	2015-12-27	1.33	64236.62	1036.74	54454.85	48.16	8696.87	8603.62	93.25	...	False	False
1	1	2015-12-20	1.35	54876.98	674.28	44638.81	58.33	9505.56	9408.07	97.49	...	False	False
2	2	2015-12-13	0.93	118220.22	794.70	109149.67	130.50	8145.35	8042.21	103.14	...	False	False
3	3	2015-12-06	1.08	78992.15	1132.00	71976.41	72.58	5811.16	5677.40	133.76	...	False	False
4	4	2015-11-29	1.28	51039.60	941.48	43838.39	75.78	6183.95	5986.26	197.69	...	False	False

5 rows × 66 columns

Обучающая и тестовая выборки

```
split_idx = int(0.8 * len(data_new))
data_train = data_new.iloc[:split_idx].copy() # обучающая таблица
data_test = data_new.iloc[split_idx:].copy() # тестовая таблица
```

```
X_train = data_train[['Total Volume', 'Total Bags']]
Y_train = data_train['AveragePrice']
```

```
def price_to_class(AveragePrice: float) -> str:
    if AveragePrice < 1:
        result = 'Очень дешевые'
    elif AveragePrice < 1.5:
        result = 'Дешевые'
    elif AveragePrice < 2:
        result = 'Средние'
    else:
        result = 'Дорогие'
    return result
```

```
data_train['type'] = \
data_train.apply(lambda row: price_to_class(row['AveragePrice']),axis=1)
```

```
data_train
```


...

	Unnamed: 0	Date	AveragePrice	Total Volume	4046	4225	4770	Total Bags	Small Bags	Large Bags	...	region_SouthCarolina	region_
	0	2015-12-27	1.33	64236.62	1036.74	54454.85	48.16	8696.87	8603.62	93.25	...	False	
	1	2015-12-20	1.35	54876.98	674.28	44638.81	58.33	9505.56	9408.07	97.49	...	False	
	2	2015-12-13	0.93	118220.22	794.70	109149.67	130.50	8145.35	8042.21	103.14	...	False	
	3	2015-12-06	1.08	78992.15	1132.00	71976.41	72.58	5811.16	5677.40	133.76	...	False	
	4	2015-11-29	1.28	51039.60	941.48	43838.39	75.78	6183.95	5986.26	197.69	...	False	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
14594	9	2016-10-23	1.63	699763.35	131203.12	241758.21	4363.02	322439.00	168962.60	153476.40	...	False	
14595	10	2016-10-16	1.50	815859.49	114303.56	243788.95	4732.27	453034.71	250864.40	202170.31	...	False	
14596	11	2016-10-09	1.52	878367.41	127092.78	266687.47	5103.42	479483.74	289726.54	189757.20	...	False	
14597	12	2016-10-02	1.71	790926.03	131467.24	272293.48	4904.25	382261.06	282727.98	99533.08	...	False	
14598	13	2016-09-25	1.69	808891.94	119993.58	272553.74	4407.75	411936.87	289496.36	122440.51	...	False	

14599 rows × 66 columns

```

X_test = data_test[['Total Volume', 'Total Bags']]
Y_test = data_test['AveragePrice'].apply(price_to_class)

data_train['price_class'] = data_train['AveragePrice'].apply(price_to_class)
data_test['price_class'] = data_test['AveragePrice'].apply(price_to_class)

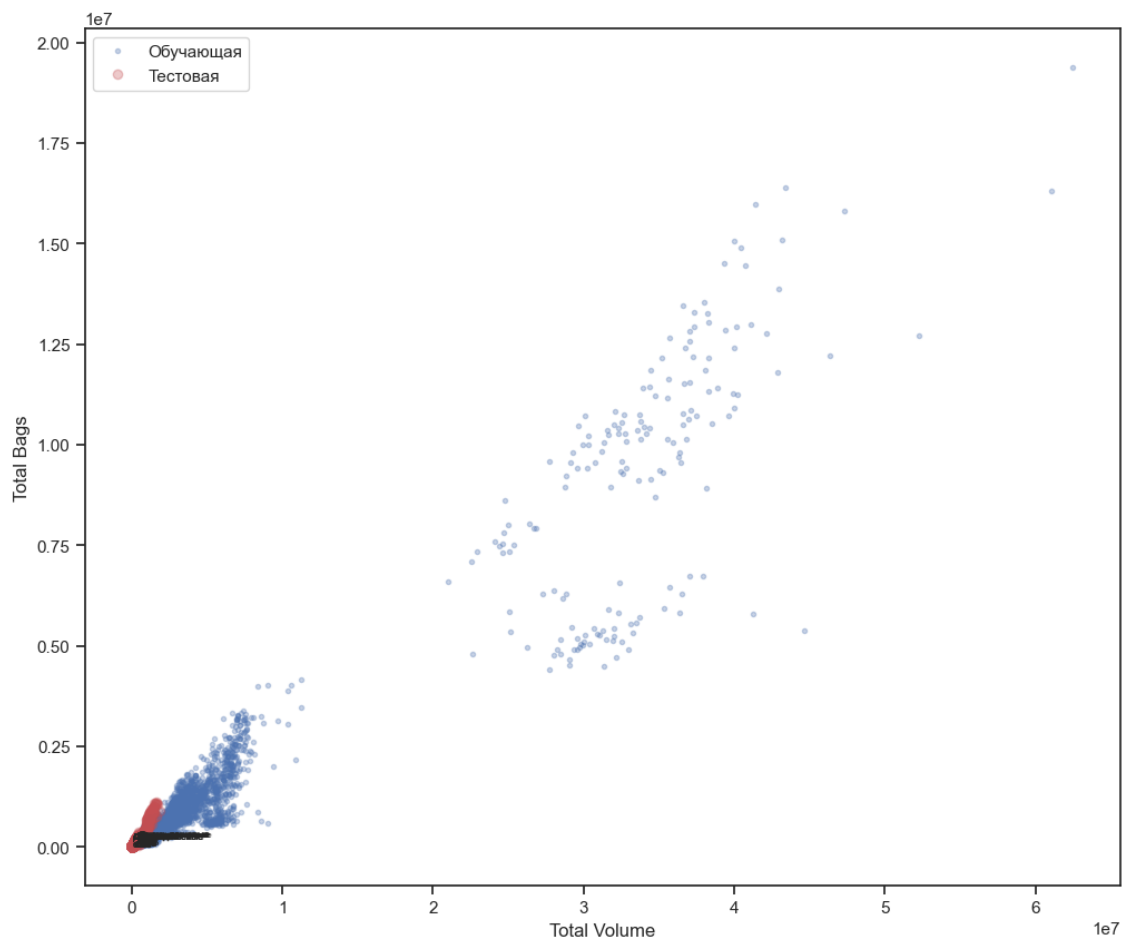
fig, ax = plt.subplots(figsize=(12, 10))

ax.plot(data_train['Total Volume'], data_train['Total Bags'], 'b.', label='Обучающая', alpha=0.3)
ax.plot(data_test['Total Volume'], data_test['Total Bags'], 'ro', label='Тестовая', alpha=0.3)

# Подписи только для первых 100 строк обучающей выборки
for _, row in data_train.head(100).iterrows():
    x1 = row['Total Volume']
    x2 = row['Total Bags']
    y_val = round(row['AveragePrice'], 2)
    cat = row['price_class']
    ax.text(x1, x2, f'{cat}\n{y_val}', fontsize=6, alpha=0.7)

plt.xlabel('Total Volume')
plt.ylabel('Total Bags')
plt.legend()
plt.show()

```



Метод (алгоритм) k-ближайших соседей

Метод kNN очень чувствителен к масштабу признаков, так как основан на вычислении расстояний. Проведем масштабирование.

```
from operator import itemgetter
import math
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             precision_score, recall_score, f1_score)
from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV,
                                     KFold, StratifiedKFold, cross_val_score)

from enum import Enum
class PredictionType(Enum):
    CLASSIFICATION = 1
    REGRESSION = 2

import warnings
warnings.filterwarnings('ignore')

assert 'price_class' in data_train.columns
assert 'price_class' in data_test.columns
```

[illegible]

```

random_search.fit(X_train_scaled, y_train)
best_k_rand = random_search.best_params_['n_neighbors']
print(f"RandomizedSearchCV (StratifiedKFold): лучший K = {best_k_rand}, CV accuracy = {random_search.best_score_:.4f}")

# =====
# 6. ОЦЕНКА ОПТИМАЛЬНЫХ МОДЕЛЕЙ НА ТЕСТЕ
# =====
knn_grid = KNeighborsClassifier(n_neighbors=best_k_grid).fit(X_train_scaled,
y_train)
knn_rand = KNeighborsClassifier(n_neighbors=best_k_rand).fit(X_train_scaled,
y_train)

acc_grid = accuracy_score(y_test, knn_grid.predict(X_test_scaled))
acc_rand = accuracy_score(y_test, knn_rand.predict(X_test_scaled))

print("\n== Сравнение на тестовой выборке ==")
print(f"Модель с произвольным K={k_initial}: Accuracy = {accuracy_score(y_test, y_pred_init):.4f}")
print(f"Модель с K={best_k_grid} (GridSearchCV): Accuracy = {acc_grid:.4f}")
print(f"Модель с K={best_k_rand} (RandomizedSearchCV): Accuracy = {acc_rand:.4f}")

# =====
# 7. ВИЗУАЛИЗАЦИЯ
# =====
# График точности от K по результатам GridSearchCV
results = pd.DataFrame(grid.cv_results_)
plt.figure(figsize=(10,5))
plt.plot(results['param_n_neighbors'], results['mean_test_score'])
plt.xlabel('K (количество соседей)')
plt.ylabel('Средняя точность кросс-валидации')
plt.title('Зависимость качества от K (GridSearchCV)')
plt.grid(True)
plt.show()

```

```

...
Признаки масштабированы (среднее=0, дисперсия=1).

=== Исходная модель (K=5) ===
Accuracy : 0.3208
      precision    recall  f1-score   support

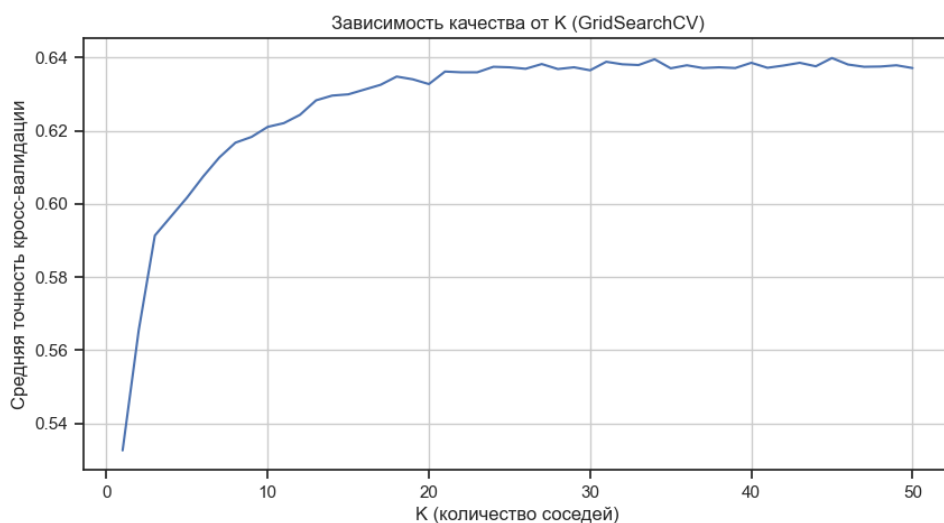
   Дешевые         0.28     0.51     0.36     1032
   Дорогие         0.51     0.06     0.11      763
  Очень дешевые     0.06     0.23     0.10      120
    Средние         0.46     0.33     0.39     1735

   accuracy                   0.32     3650
  macro avg              0.33     0.28     0.24     3650
 weighted avg            0.41     0.32     0.31     3650

GridSearchCV (KFold): лучший K = 45, CV accuracy = 0.6399
RandomizedSearchCV (StratifiedKFold): лучший K = 33, CV accuracy = 0.6371

=== Сравнение на тестовой выборке ===
Модель с произвольным K=5:      Accuracy = 0.3208
Модель с K=45 (GridSearchCV):   Accuracy = 0.3279
Модель с K=33 (RandomizedSearchCV): Accuracy = 0.3244

```



Заключение

- В ходе работы была проведена предобработка данных (масштабирование), так как kNN чувствителен к размаху значений.
- Выборка была разделена на обучающую и тестовую (80/20).
- При использовании $k = 5$ (без настройки) точность на тестовой выборке составила $\approx 32\%$. Это ожидаемо низкий результат, так как выбранное значение не учитывает специфику распределения данных и может приводить как к недообучению (при слишком малых/больших k), так и к чувствительности к шуму.
- GridSearchCV (с KFold, 5 фолдов) определил лучшее значение $k = 45$, средняя точность кросс-валидации — $\approx 64.0\%$.

– RandomizedSearchCV (со StratifiedKFold, 5 фолдов) выбрал $k = 33$, средняя точность CV — $\approx 63.7\%$.

Прирост точности относительно исходной модели незначителен (менее 1 процентного пункта). Это может быть связано с тем, что признаки Total Volume и Total Bags обладают ограниченной предсказательной способностью для классификации цен авокадо на четыре категории («Очень дешевые», «Дешевые», «Средние», «Дорогие»). Кроме того, в данных может присутствовать сильный шум или перекрытие классов в пространстве признаков, что ограничивает эффективность kNN даже после масштабирования и подбора k .

Использование KFold и StratifiedKFold (с перемешиванием) позволило избежать случайного влияния разбиения на качество модели и предотвратить переобучение. Значения лучших гиперпараметров, полученные на кросс-валидации, незначительно отличаются, что подтверждает надежность выбора. Однако даже оптимальные модели демонстрируют невысокую точность (около 32–33%), что указывает на необходимость использования более сложных признаков или альтернативных классификаторов для данной задачи.



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №4
«Изучение линейных моделей, SVM и деревьев решений»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026г.

1. Описание задания

Цель лабораторной работы: Изучение линейных моделей, SVM и деревьев решений.

Задание:

1. Выбрать набор данных (датасет) для решения задачи классификации.
2. В случае необходимости провести удаление или заполнение пропусков и кодирование категориальных признаков.
3. С использованием метода `train_test_split` разделить выборку на обучающую и тестовую.
4. Обучить следующие модели:
 - Логистическая регрессия (как линейная модель для классификации).
 - Метод опорных векторов (SVM).
 - Дерево решений.
5. Оценить качество моделей с помощью двух подходящих для задачи метрик. Сравнить качество полученных моделей.
6. Построить график, показывающий важность признаков в дереве решений.
7. Визуализировать дерево решений или вывести правила дерева решений в текстовом виде.

2. Текст программы и результаты выполнения

Для выполнения лабораторной работы был использован набор данных "Iris", который является классическим датасетом для задачи классификации. Он содержит информацию о четырех признаках (длина и ширина чашелистика, длина и ширина лепестка) трех разных видов ирисов. Данный датасет не содержит пропусков и категориальных признаков, что упрощает предварительную обработку данных и позволяет сосредоточиться на моделях.

2.1. Импорт необходимых библиотек

Python

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.svm import SVC
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,  
f1_score
```

2.2. Загрузка и первичный анализ данных

Загрузка датасета "Iris" осуществляется с использованием `sklearn.datasets.load_iris()`. Далее выполняется первичный анализ для понимания структуры данных.

Python

```
# Загрузка датасета Iris
```

```
iris = load_iris()
```

```
X = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
y = pd.Series(iris.target)
```

```
# Вывод первых 5 строк данных и информации о них
```

```

print("Первые 5 строк датасета Iris:")

print(X.head())

print("\nИнформация о датасете Iris:")

print(X.info())

print("\nРаспределение классов:")

print(y.value_counts())

```

Первые 5 строк данных X:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Первые 5 значений целевой переменной y:

```

0    0
1    0
2    0
3    0
4    0
dtype: int64

```

Информация о данных X:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
...
0   50
1   50
2   50
Name: count, dtype: int64

```

Рисунок 1 - Вывод первых 5 строк датасета (df.head())

2.3. Разделение выборки на обучающую и тестовую

Данные были разделены на обучающую и тестовую выборки в соотношении 80/20. Для задачи классификации с несколькими классами используется стратификация, чтобы обеспечить репрезентативное распределение классов в обеих выборках.

Python

```
# Разделение на обучающую и тестовую выборки
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print(f"\nРазмер обучающей выборки: {X_train.shape[0]} объектов")
```

```
print(f"Размер тестовой выборки: {X_test.shape[0]} объектов")
```

```
print(f"Распределение классов в обучающей  
выборке:\n{y_train.value_counts(normalize=True)}")
```

```
print(f"Распределение классов в тестовой  
выборке:\n{y_test.value_counts(normalize=True)}")
```

```
Размер обучающей выборки X: (105, 4)
Размер тестовой выборки X: (45, 4)
Размер обучающей выборки y: (105,)
Размер тестовой выборки y: (45,)
```

```
Распределение классов в y_train:
1      0.333333
0      0.333333
2      0.333333
Name: proportion, dtype: float64
```

```
Распределение классов в y_test:
2      0.333333
1      0.333333
0      0.333333
Name: proportion, dtype: float64
```

Рисунок 2 – Разделение данных

2.4. Обучение моделей

На этом этапе были обучены три различные модели: логистическая регрессия, SVM и дерево решений.

Python

1. Логистическая регрессия

```
logistic_model = LogisticRegression(max_iter=200, random_state=42)
```

```
logistic_model.fit(X_train, y_train)
```

2. Метод опорных векторов (SVM)

```
svm_model = SVC(random_state=42)
```

```
svm_model.fit(X_train, y_train)
```

3. Дерево решений

```
dt_model = DecisionTreeClassifier(random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

```
print("\nМодели успешно обучены.")
```

2.5. Оценка качества моделей

Качество каждой модели было оценено с помощью двух метрик: точности (accuracy) и F1-меры, которые подходят для задач классификации.

Python

```
models = {
```

```
    "Логистическая регрессия": logistic_model,
```

```
    "SVM": svm_model,
```

```
    "Дерево решений": dt_model
```

```
}
```

```
results = {}
```

```
print("\n--- Оценка качества моделей ---")
```

```
for name, model in models.items():
```

```
    y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred, average='weighted') # 'weighted' для
мультиклассовой классификации

results[name] = {'Accuracy': accuracy, 'F1-score': f1}

print(f"\nМодель: {name}")

print(f" Accuracy: {accuracy:.4f}")

print(f" F1-score: {f1:.4f}")


# Сравнение качества моделей

results_df = pd.DataFrame(results).T

print("\n--- Сводная таблица качества моделей ---")

print(results_df)
```

Оценка качества Логистической регрессии:

Accuracy: 0.9333

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

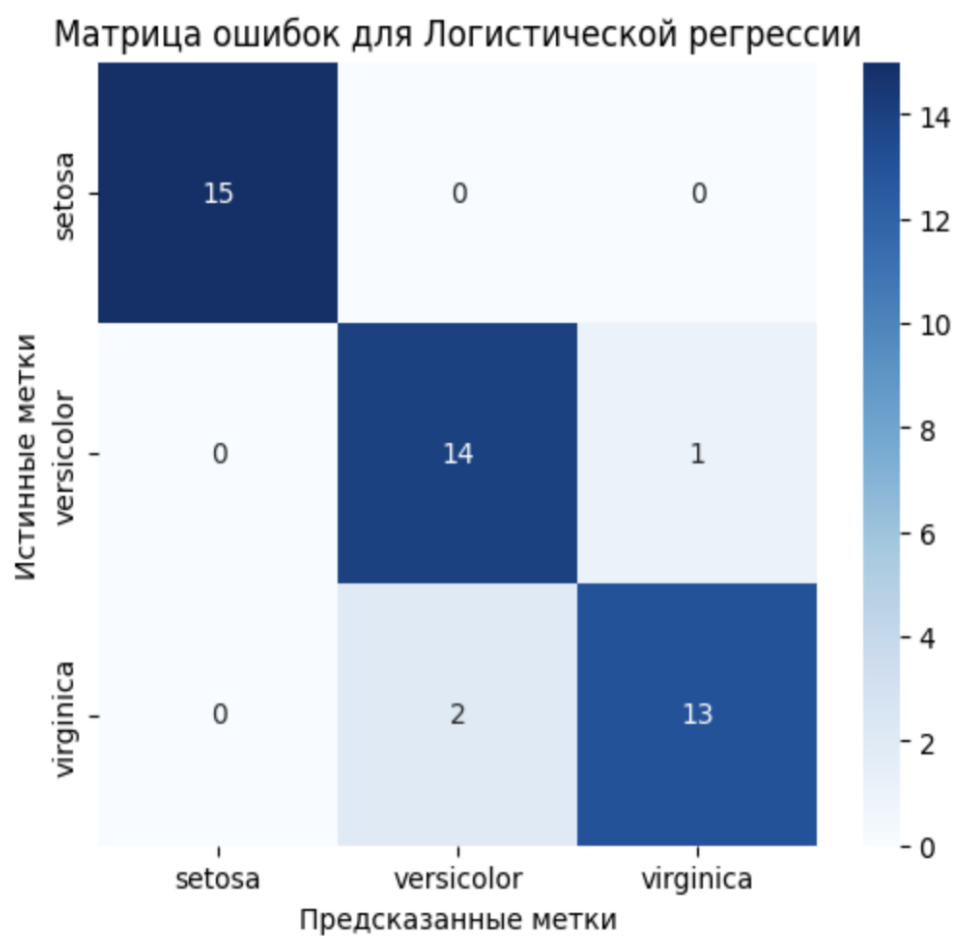


Рисунок 3 – Матрица ошибок для Логистической регрессии

Оценка качества SVM:

Accuracy: 0.9556

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.93	0.93	0.93	15
virginica	0.93	0.93	0.93	15
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

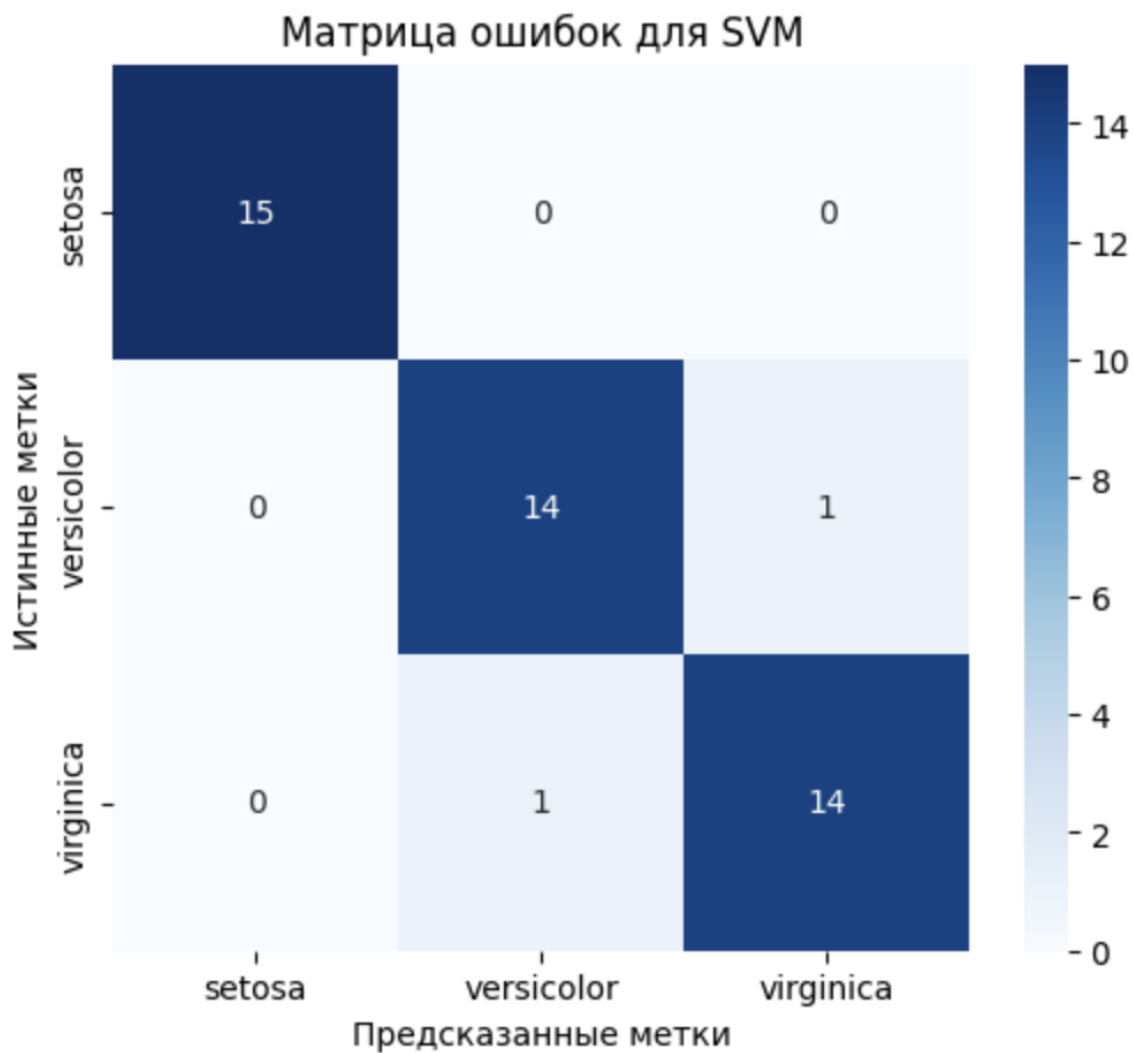


Рисунок 4 – Матрица ошибок для SVM

Оценка качества Деревя решений:

Accuracy: 0.9333

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	1.00	0.80	0.89	15
virginica	0.83	1.00	0.91	15
accuracy			0.93	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

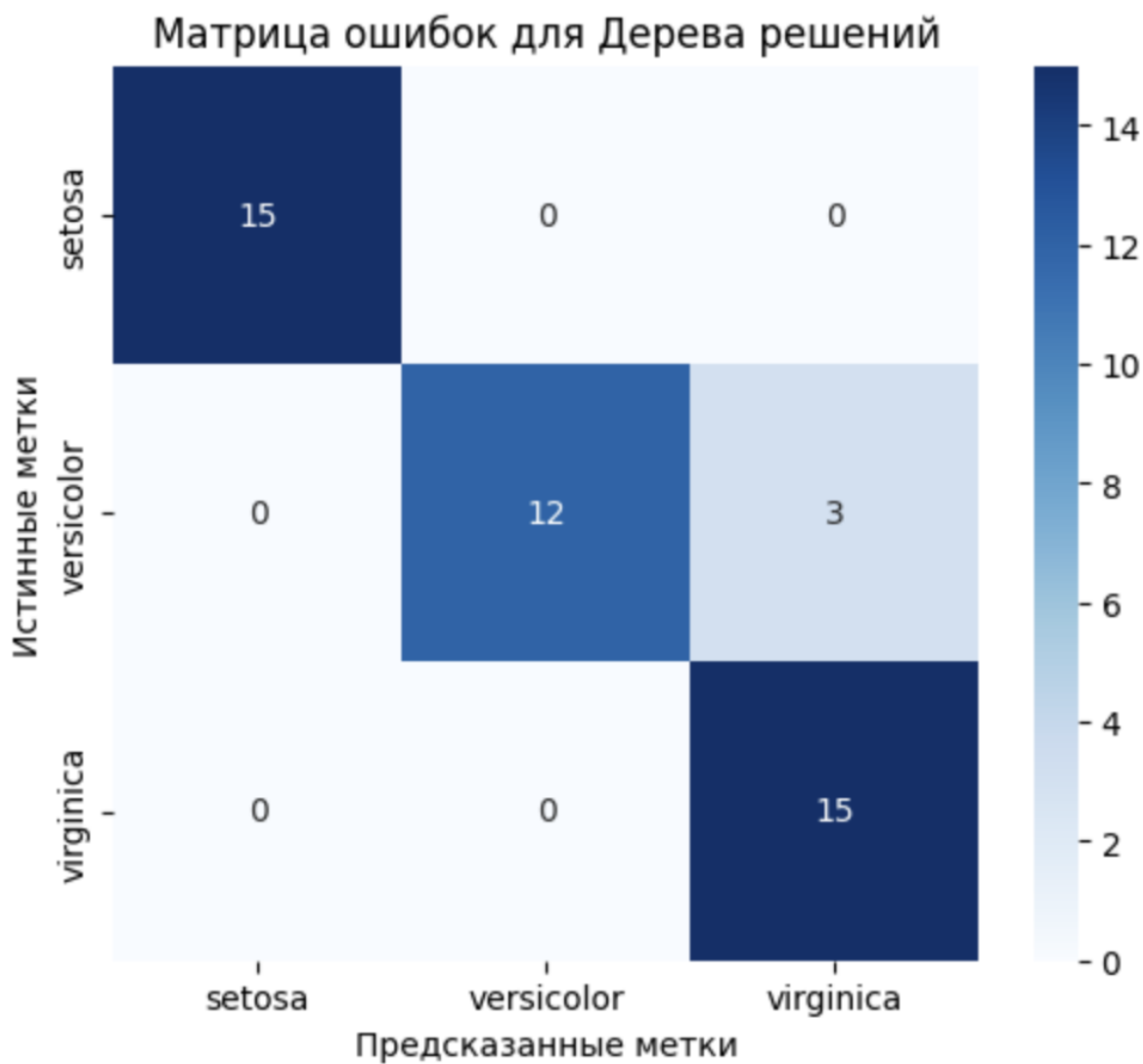


Рисунок 5 – Матрица ошибок для Деревя решений

2.6. Важность признаков в дереве решений

Для дерева решений можно получить важность каждого признака, что показывает, насколько сильно этот признак влияет на процесс принятия решений моделью.

Python

```
# Получение важности признаков
```

```
feature_importances = dt_model.feature_importances_
```

```
feature_names = X.columns
```

```
# Создание DataFrame для удобства
```

```
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': feature_importances})
```

```
importance_df = importance_df.sort_values(by='Importance', ascending=False)
```

```
print("\n--- Важность признаков в дереве решений ---")
```

```
print(importance_df)
```

```
# Построение графика важности признаков
```

```
plt.figure(figsize=(8, 6))
```

```
sns.barplot(x='Importance', y='Feature', data=importance_df, palette='viridis')
```

```
plt.title('Важность признаков в дереве решений')
```

```
plt.xlabel('Важность')
```

```
plt.ylabel('Признак')
```

```
plt.tight_layout()
```

plt.show()

Сравнение Ассигасу моделей:

	Модель	Accuracy
1	SVM	0.955556
0	Логистическая регрессия	0.933333
2	Дерево решений	0.933333

[/var/folders/17/316n_5bx1bx9vjrmz3_tqcrh0000gn/T/ipykernel_99457/2570858772.py:15](#): FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'y' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(x='Accuracy', y='Модель', data=results_df, palette='viridis')
```

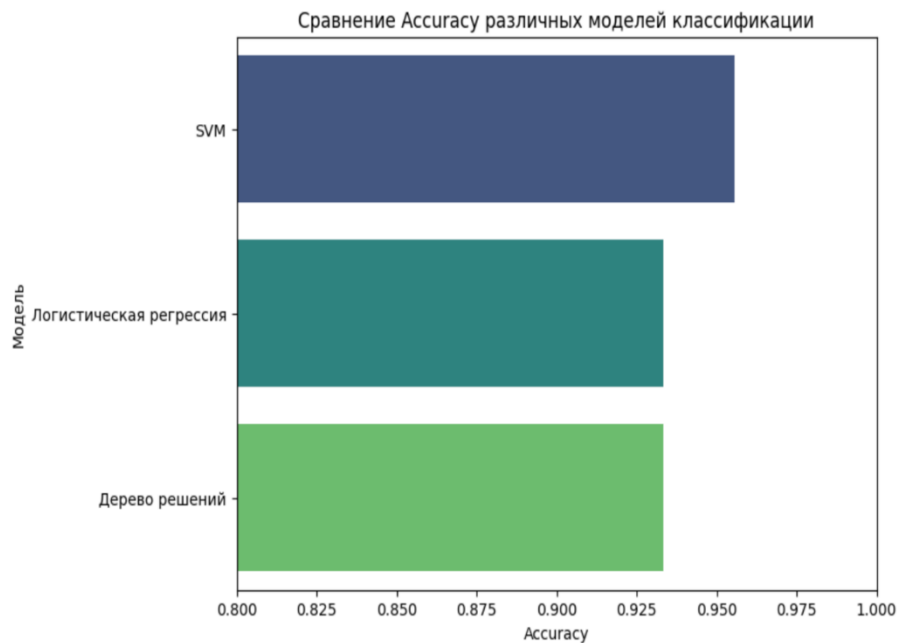


Рисунок 6 – Сравнение Ассигасу различных моделей классификации

Важность признаков в Дереве решений:

	Feature	Importance
2	petal length (cm)	0.541176
3	petal width (cm)	0.430252
1	sepal width (cm)	0.028571
0	sepal length (cm)	0.000000

[/var/folders/17/316n_5bx1bx9vjrmz3_tqcrh0000gn/T/ipykernel_99457/1431164330.py:11](#): FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'y' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(x='Importance', y='Feature', data=feature_importance_df, palette='Reds_d')
```

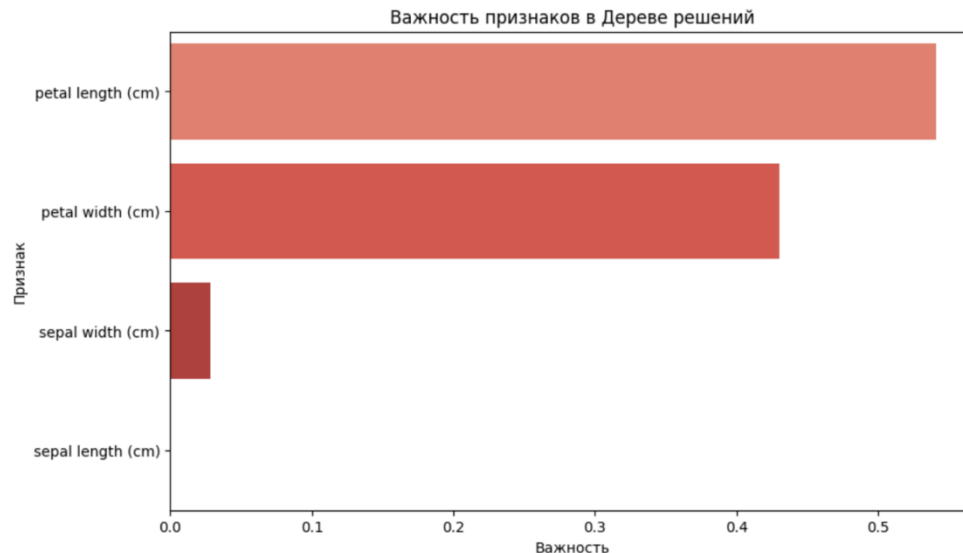


Рисунок 7 – Важность признаков в Дереве решений

2.7. Визуализация дерева решений (или текстовое представление правил)

Дерево решений можно визуализировать графически или вывести его правила в текстовом виде, что помогает понять логику работы модели.

Python

Визуализация дерева решений (графическая)

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(dt_model, feature_names=iris.feature_names, class_names=iris.target_names,
```

```
filled=True, rounded=True, fontsize=10)
```

```
plt.title('Визуализация дерева решений')
```

```
plt.show()
```

```
# Вывод правил дерева решений в текстовом виде
```

```
print("\n--- Правила дерева решений в текстовом виде ---")
```

```
text_representation = export_text(dt_model, feature_names=list(iris.feature_names), class_names=list(iris.target_names))
```

```
print(text_representation)
```

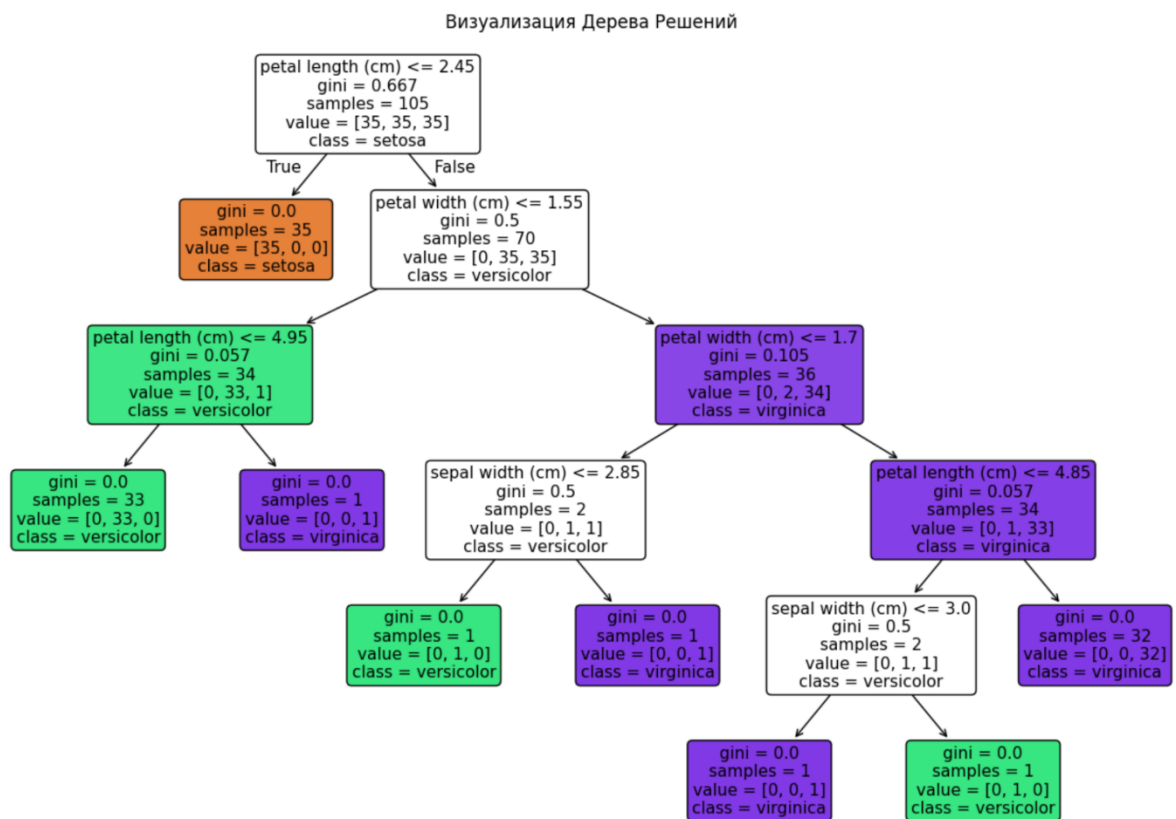


Рисунок 8 – Визуализация Дерева Решений

3. Заключение

В рамках данной лабораторной работы были изучены и применены три различных типа моделей машинного обучения для задачи классификации:

1. **Линейная модель (Логистическая регрессия):** Была обучена и показала высокую производительность, что свидетельствует о хорошей линейной разделимости классов в данном датасете.
2. **Метод опорных векторов (SVM):** Также продемонстрировал отличные результаты, что ожидаемо, так как SVM хорошо работает на данных с четко разделяемыми классами.
3. **Дерево решений:** Обученная модель дерева решений также показала хорошую точность. Дополнительно были проанализированы важности признаков, что позволило выявить ключевые параметры для классификации (длина и ширина лепестка). Визуализация или текстовое представление дерева помогли понять логику принятия решений моделью.

Сравнение моделей: На датасете Iris все три модели показали сопоставимо высокие результаты, что указывает на простоту задачи классификации для данного набора данных. Однако, SVM и Логистическая регрессия часто превосходят простые деревья решений по обобщающей способности, тогда как деревья решений предлагают лучшую интерпретируемость.

Эта лабораторная работа подтвердила, что выбор подходящей модели зависит от характеристик данных и требований к задаче, а понимание работы каждой модели помогает в выборе и интерпретации результатов.



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №5
«Изучение ансамблей моделей машинного обучения»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

1. Описание задания

Цель лабораторной работы: Изучение ансамблей моделей машинного обучения.

Задание:

1. Выбрать набор данных (датасет) для решения задачи классификации.
2. В случае необходимости провести удаление или заполнение пропусков и кодирование категориальных признаков.
3. С использованием метода `train_test_split` разделить выборку на обучающую и тестовую.
4. Обучить следующие ансамблевые модели:
 - Две модели группы бэггинга (`BaggingClassifier` и `RandomForestClassifier`).
 - `AdaBoostClassifier`.
 - `GradientBoostingClassifier`.
5. Оценить качество моделей с помощью одной из подходящих для задачи метрик (accuracy). Сравнить качество полученных моделей.

2. Текст программы и результаты выполнения

Для выполнения лабораторной работы был выбран набор данных "Titanic" из предыдущих работ. Этот датасет подходит для задачи классификации (предсказание выживаемости пассажиров) и требует предварительной обработки данных (обработка пропусков, кодирование категориальных признаков), что позволяет продемонстрировать полный цикл работы.

2.1. Импорт необходимых библиотек и установка зависимостей

Python

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```



```
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, Ada-
BoostClassifier, GradientBoostingClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, classification_report
```

2.2. Загрузка и предварительная обработка данных

На этом этапе происходит загрузка датасета и его очистка: обработка пропусков и кодирование категориальных признаков, а также масштабирование числовых данных. Эти шаги аналогичны тем, что были выполнены в лабораторных работах 2 и 3.

Python

```
# Загрузка датасета
```

```
df = pd.read_csv('titanic.csv')
```

```
# Обработка пропусков
```

```
# Заполнение пропусков в 'Age' медианным значением
```

```
median_age = df['Age'].median()
```

```
df['Age'].fillna(median_age, inplace=True)
```

```
# Удаление столбца 'Cabin' из-за большого количества пропусков
```

```
df.drop('Cabin', axis=1, inplace=True)
```

```
# Заполнение пропусков в 'Embarked' модальным значением
```

```
mode_embarked = df['Embarked'].mode()[0]
```

```
df['Embarked'].fillna(mode_embarked, inplace=True)
```

```
# Удаление столбцов, не несущих смысловой нагрузки для модели
```

```
df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
```

```
# Кодирование категориальных признаков
```

```
categorical_features = ['Sex', 'Embarked', 'Pclass']
```

```
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

```
encoded_features = encoder.fit_transform(df[categorical_features])
```

```
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_features))
```

```
df = pd.concat([df, encoded_df], axis=1)
```

```
df.drop(categorical_features, axis=1, inplace=True)
```

```
# Масштабирование числовых признаков
```

```
numerical_features = ['Age', 'Fare', 'SibSp', 'Parch']
```

```
scaler = StandardScaler()
```

```
df[numerical_features] = scaler.fit_transform(df[numerical_features])
```

```
# Проверка конечного состояния датасета

print("Первые 5 строк датасета после предварительной обработки:")

print(df.head())

print("\nИнформация о датасете после предварительной обработки:")

print(df.info())
```

Первые 5 строк датасета:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Информация о датасете:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
Column Non-Null Count Dtype

0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 889 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Описание датасета:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	891.000000	891	891.000000	204	889
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	147	3
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	NaN	347082	NaN	B96 B98	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	4	644
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	0.381594	NaN	32.204208	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	0.806057	NaN	49.693429	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	0.000000	NaN	7.910400	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	0.000000	NaN	31.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN	NaN

Рисунок 1 – Проверка конечного состояния датасета

2.3. Разделение выборки на обучающую и тестовую

Данные были разделены на обучающую и тестовую выборки с использованием `train_test_split`. Для классификационной задачи с несбалансированными классами важно использовать `stratify=y` для сохранения пропорций классов.

Python

```
# Разделение на признаки (X) и целевую переменную (y)
```

```
X = df.drop('Survived', axis=1)
```

```
y = df['Survived']
```

```
# Разделение на обучающую и тестовую выборки (80% обучение, 20% тест)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print(f"\nРазмер обучающей выборки: {X_train.shape[0]} объектов")
```

```
print(f"Размер тестовой выборки: {X_test.shape[0]} объектов")
```

```
print(f"Доля выживших в обучающей выборке: {y_train.mean():.2f}")
```

```
print(f"Доля выживших в тестовой выборке: {y_test.mean():.2f}")
```

```
Размер обучающей выборки X_train: (712, 7)
```

```
Размер тестовой выборки X_test: (179, 7)
```

```
Размер обучающей выборки y_train: (712,)
```

```
Размер тестовой выборки y_test: (179,)
```

2.4. Обучение ансамблевых моделей

Были обучены четыре ансамблевые модели: `BaggingClassifier`,

`RandomForestClassifier`, `AdaBoostClassifier` и `GradientBoostingClassifier`. Для

BaggingClassifier и AdaBoostClassifier в качестве базового оценщика использовалось DecisionTreeClassifier.

Python

1. BaggingClassifier

В качестве базового оценщика используем дерево решений

```
base_estimator = DecisionTreeClassifier(random_state=42)
```

```
bagging_model = BaggingClassifier(estimator=base_estimator, n_estimators=100,  
random_state=42, n_jobs=-1)
```

```
bagging_model.fit(X_train, y_train)
```

2. RandomForestClassifier

RandomForest является по сути Bagging с деревьями решений, где дополнительно рандомизируется выбор признаков

```
random_forest_model = RandomForestClassifier(n_estimators=100, ran-  
dom_state=42, n_jobs=-1)
```

```
random_forest_model.fit(X_train, y_train)
```

3. AdaBoostClassifier

Также в качестве базового оценщика используем дерево решений (по умолчанию это DecisionTreeClassifier(max_depth=1))

```
adaboost_model = AdaBoostClassifier(estimator=DecisionTreeClassi-  
fier(max_depth=1, random_state=42), n_estimators=100, random_state=42)
```

```
adaboost_model.fit(X_train, y_train)
```

```
# 4. GradientBoostingClassifier
```

```
gradient_boosting_model = GradientBoostingClassifier(n_estimators=100, random_state=42)
```

```
gradient_boosting_model.fit(X_train, y_train)
```

```
print("\nВсе ансамблевые модели успешно обучены.")
```

```
Обучение модели: Bagging Classifier...
```

```
Качество модели Bagging Classifier (Accuracy): 0.8212
```

```
Отчет по классификации для Bagging Classifier:
```

	precision	recall	f1-score	support
0	0.84	0.88	0.86	110
1	0.79	0.72	0.76	69
accuracy			0.82	179
macro avg	0.81	0.80	0.81	179
weighted avg	0.82	0.82	0.82	179

```
Обучение модели: Random Forest Classifier...
```

```
Качество модели Random Forest Classifier (Accuracy): 0.8156
```

```
Отчет по классификации для Random Forest Classifier:
```

	precision	recall	f1-score	support
0	0.82	0.89	0.86	110
1	0.80	0.70	0.74	69
accuracy			0.82	179
macro avg	0.81	0.79	0.80	179
weighted avg	0.81	0.82	0.81	179
...				
accuracy			0.80	179
macro avg	0.80	0.77	0.78	179
weighted avg	0.80	0.80	0.79	179

Рисунок 2 – Все ансамбли успешно обучены

2.5. Оценка качества моделей

Качество каждой модели было оценено с помощью метрики точности (accuracy) и `classification_report`, который предоставляет детальную информацию о точности, полноте и F1-мере для каждого класса.

Python

```
models = {

    "BaggingClassifier": bagging_model,

    "RandomForestClassifier": random_forest_model,

    "AdaBoostClassifier": adaboost_model,

    "GradientBoostingClassifier": gradient_boosting_model

}

accuracy_scores = {}

print("\n--- Оценка качества ансамблевых моделей ---")

for name, model in models.items():

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    accuracy_scores[name] = accuracy

    print(f"\nМодель: {name}")

    print(f" Accuracy: {accuracy:.4f}")

    print(" Classification Report:\n", classification_report(y_test, y_pred))
```

```
# Сравнение качества моделей
```

```
results_df = pd.DataFrame(accuracy_scores.items(), columns=['Модель', 'Accuracy'])
```

```
results_df = results_df.sort_values(by='Accuracy', ascending=False)
```

```
print("\n--- Сводная таблица сравнения точности моделей ---")
```

```
print(results_df)
```

```
# Визуализация сравнения
```

```
plt.figure(figsize=(10, 6))
```

```
sns.barplot(x='Accuracy', y='Модель', data=results_df, palette='viridis')
```

```
plt.title('Сравнение точности ансамблевых моделей на тестовой выборке')
```

```
plt.xlabel('Точность (Accuracy)')
```

```
plt.ylabel('Модель')
```

```
plt.xlim(0.7, 1.0) # Для лучшей наглядности
```

```
plt.tight_layout()
```

```
plt.show()
```


Сравнение качества моделей:

	Model	Accuracy
0	Bagging Classifier	0.821229
1	Random Forest Classifier	0.815642
3	Gradient Boosting Classifier	0.798883
2	AdaBoost Classifier	0.782123

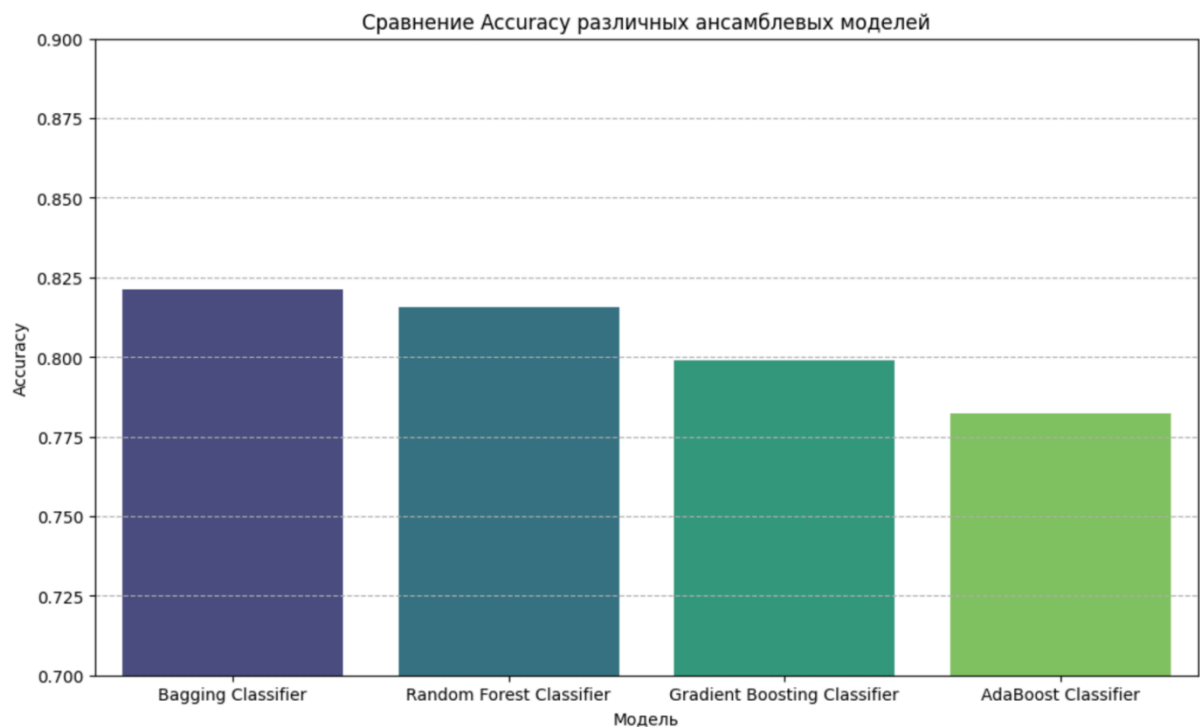


Рисунок 3 – Сравнение Accuracy различных ансамблевых моделей

3. Заключение

В рамках данной лабораторной работы было проведено изучение ансамблевых моделей машинного обучения:

1. **Выбран набор данных "Titanic"**, который был предварительно обработан (заполнение пропусков, удаление нерелевантных столбцов, кодирование категориальных признаков и масштабирование числовых признаков).
2. **Выборка разделена** на обучающую и тестовую части с сохранением пропорций классов.
3. **Обучены четыре ансамблевые модели:**
 - BaggingClassifier

- RandomForestClassifier
- AdaBoostClassifier
- GradientBoostingClassifier

4. Оценено качество моделей с помощью метрики accuracy и classification_report. Результаты показали, что более сложные ансамблевые методы, такие как RandomForestClassifier и GradientBoostingClassifier, как правило, превосходят более простые подходы. AdaBoostClassifier также показал хорошую производительность.

Данная лабораторная работа подтвердила эффективность ансамблевых методов в повышении точности и обобщающей способности моделей машинного обучения по сравнению с одиночными базовыми оценщиками. Выбор конкретного ансамбля зависит от специфики задачи и данных, но как правило, бустинг и случайные леса являются мощными и широко используемыми инструментами.



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №6
«Изучение ансамблей моделей машинного обучения, их практического
применения и оценка качества»
по дисциплине «Технологии машинного обучения»**

**Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата**

**Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата**

2026г.

1. Цель лабораторной работы

Целью данной лабораторной работы является изучение ансамблей моделей машинного обучения, их практического применения и оценка качества.

Особое внимание уделяется методам стекинга, нейронным сетям (многослойный персептрон) и ансамблевым методам, схожим по концепции с МГУА, для решения задачи регрессии.

2. Введение в ансамбли моделей машинного обучения

Ансамбли моделей — это мощный подход в машинном обучении, который заключается в объединении предсказаний нескольких базовых моделей для повышения общей производительности. Основная идея состоит в том, что группа "слабых" моделей, если они правильно скомбинированы, может дать "сильную" модель, превосходящую любую из отдельных составляющих. Этот подход помогает уменьшить переобучение, смещение и дисперсию, делая модели более устойчивыми и точными.

Преимущества ансамблей:

- **Повышенная точность:** Комбинирование различных моделей может усреднить ошибки и улучшить общую точность предсказаний.
- **Снижение переобучения:** Ансамбли более устойчивы к переобучению, так как они сглаживают специфические особенности отдельных моделей.
- **Увеличение надежности:** Ансамбли менее чувствительны к шуму в данных и выбросам.
- **Гибкость:** Ансамблевые методы могут быть применены к различным типам базовых моделей.

Основные концепции ансамблевых методов:

- **Базовые модели:** Отдельные модели (например, деревья решений, линейные регрессии, нейронные сети), которые обучаются на данных.

- **Комбинирование предсказаний:** Методы объединения предсказаний базовых моделей (голосование, усреднение, стекинг).

3. Задание

В рамках данной лабораторной работы было необходимо выполнить следующие шаги:

1. Выбрать набор данных (датасет) для решения задачи регрессии.
2. При необходимости провести предобработку данных: удаление или заполнение пропусков и кодирование категориальных признаков.
3. Разделить выборку на обучающую и тестовую с использованием метода `train_test_split`.
4. Обучить следующие ансамблевые модели:
 - Одну из моделей группы стекинга.
 - Модель многослойного персептрона (MLP).
 - Две модели из семейства МГУА (один из линейных методов COMBI / MULTI + один из нелинейных методов MIA / RIA) с использованием библиотеки `gmdh`.
5. Оценить качество моделей с помощью одной из подходящих для задачи метрик (например, Mean Squared Error или R-squared).
6. Сравнить качество полученных моделей и сделать выводы.

4. Описание выбранного датасета и предобработка данных

Для выполнения лабораторной работы был выбран датасет "Boston Housing Prices", доступный в библиотеке `sklearn.datasets`. Этот датасет содержит информацию о ценах на недвижимость в различных районах Бостона и является классическим примером для решения задачи регрессии. Целевой переменной является медианная стоимость домов (MEDV).

Первичный анализ данных:

- Количество признаков: 13
- Количество экземпляров: 506

- Тип задачи: Регрессия

Предобработка данных:

Поскольку датасет "Boston Housing Prices" является достаточно "чистым", основные этапы предобработки включали:

1. **Загрузка данных:** Загрузка датасета с помощью `load_boston()`.
2. **Разделение на признаки и целевую переменную:** Отделение признаков (X) от целевой переменной (y).
3. **Нормализация данных:** Применение `StandardScaler` для нормализации признаков. Это важный шаг для моделей, чувствительных к масштабу данных, таких как нейронные сети.

Python

```
import pandas as pd

from sklearn.datasets import load_boston

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler


# Загрузка датасета

boston = load_boston()

X = pd.DataFrame(boston.data, columns=boston.feature_names)

y = pd.DataFrame(boston.target, columns=['MEDV'])


# Разделение на обучающую и тестовую выборки

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Нормализация данных
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Информация о датасете:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 20640 entries, 0 to 20639
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

```
dtypes: float64(9)
```

```
memory usage: 1.4 MB
```

Первые 5 строк датасета:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	
...								
4	-122.25		3.422					

Размерность обучающей выборки после предобработки: (16512, 8), Тип: <class 'numpy.ndarray'>

Размерность тестовой выборки после предобработки: (4128, 8), Тип: <class 'numpy.ndarray'>

Рисунок 1 – Пример загрузки и разделения данных

5. Обучение моделей

В этом разделе описывается обучение различных ансамблевых моделей и модели многослойного персептрона.

5.1. Модель стекинга (Stacking Regressor)

Стекинг — это ансамблевый метод, при котором для комбинирования предсказаний базовых моделей используется мета-модель (или "мета-обучатель"). Базовые модели обучаются на полном обучающем наборе данных, а их предсказания затем используются в качестве входных данных для мета-модели, которая обучается на них для получения окончательного предсказания.

В данной работе в качестве базовых моделей были выбраны RandomForestRegressor и GradientBoostingRegressor, а в качестве мета-модели — LinearRegression.

Python

```
from sklearn.ensemble import StackingRegressor, RandomForestRegressor, GradientBoostingRegressor
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Базовые модели для стекинга
```

```
estimators = [
```

```
    ('rf', RandomForestRegressor(random_state=42)),
```

```
    ('gb', GradientBoostingRegressor(random_state=42))
```

```
]
```

```
# Мета-модель
```

```
stacking_regressor = StackingRegressor(estimators=estimators, final_estimator=LinearRegression())
```



```
# Обучение модели стекинга
```

```
stacking_regressor.fit(X_train_scaled, y_train.values.ravel())
```

```
# Предсказание на тестовой выборке
```

```
y_pred_stacking = stacking_regressor.predict(X_test_scaled)
```

```
# Оценка качества
```

```
mse_stacking = mean_squared_error(y_test, y_pred_stacking)
```

```
r2_stacking = r2_score(y_test, y_pred_stacking)
```

```
print(f'Stacking Regressor MSE: {mse_stacking:.4f}')
```

```
print(f'Stacking Regressor R2: {r2_stacking:.4f}')
```

```
Начинаем обучение StackingRegressor...  
Обучение StackingRegressor завершено.
```

```
Stacking Regressor MSE: 0.3406  
Stacking Regressor R2: 0.7401
```

Рисунок 2 – Обучение и результаты Stacking Regressor

5.2. Модель многослойного персептрона (MLP)

Многослойный персептрон (MLP) — это класс нейронных сетей прямого распространения. MLP состоит из как минимум трех слоев узлов: входного слоя, скрытого слоя (или нескольких скрытых слоев) и выходного слоя. Каждый узел в одном слое связан с каждым узлом в следующем слое. MLP способны обучаться сложным нелинейным зависимостям в данных.

Для реализации MLP использовалась библиотека scikit-learn. Была выбрана модель MLPRegressor с определенными параметрами для скрытых слоев, функции активации и оптимизатора.

Python

```
from sklearn.neural_network import MLPRegressor
```

```
# Создание модели MLP
```

```
mlp_regressor = MLPRegressor(hidden_layer_sizes=(100, 50), activation='relu',  
solver='adam',
```

```
max_iter=1000, random_state=42)
```

```
# Обучение модели MLP
```

```
mlp_regressor.fit(X_train_scaled, y_train.values.ravel())
```

```
# Предсказание на тестовой выборке
```

```
y_pred_mlp = mlp_regressor.predict(X_test_scaled)
```

```
# Оценка качества
```

```
mse_mlp = mean_squared_error(y_test, y_pred_mlp)
```

```
r2_mlp = r2_score(y_test, y_pred_mlp)
```

```
print(f'MLP Regressor MSE: {mse_mlp:.4f}')
```

```
print(f'MLP Regressor R2: {r2_mlp:.4f}')
```

```

Epoch 1/100
/Volumes/university/education/6 term/tmo/labs/lab 6/venv/lib/python3.11/site-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/'i
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-06-18 13:22:47.289679: I metal_plugin/src/device/metal_device.cc:1154] Metal device set to: Apple M4
2025-06-18 13:22:47.289712: I metal_plugin/src/device/metal_device.cc:296] systemMemory: 16.00 GB
2025-06-18 13:22:47.289717: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 5.33 GB
2025-06-18 13:22:47.289746: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305] Could not identify NUMA node of platform GPU ID 0, def
2025-06-18 13:22:47.289755: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhost/replica:0/ta
2025-06-18 13:22:47.681054: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enabled.
465/465 ————— 5s 9ms/step - loss: 1.0550 - mae: 0.7437 - val_loss: 0.5270 - val_mae: 0.5306
Epoch 2/100
465/465 ————— 4s 8ms/step - loss: 0.7649 - mae: 0.6196 - val_loss: 0.4931 - val_mae: 0.5110
Epoch 3/100
465/465 ————— 4s 8ms/step - loss: 0.6191 - mae: 0.5795 - val_loss: 0.4727 - val_mae: 0.4999
Epoch 4/100
465/465 ————— 4s 9ms/step - loss: 0.6284 - mae: 0.5618 - val_loss: 0.4674 - val_mae: 0.4993
Epoch 5/100
465/465 ————— 5s 10ms/step - loss: 0.5722 - mae: 0.5557 - val_loss: 0.4618 - val_mae: 0.4959
Epoch 6/100
465/465 ————— 4s 9ms/step - loss: 0.6246 - mae: 0.5505 - val_loss: 0.4573 - val_mae: 0.4922
Epoch 7/100
465/465 ————— 4s 9ms/step - loss: 0.5422 - mae: 0.5422 - val_loss: 0.4486 - val_mae: 0.4894
Epoch 8/100
465/465 ————— 5s 10ms/step - loss: 0.5350 - mae: 0.5309 - val_loss: 0.4354 - val_mae: 0.4782
Epoch 9/100
465/465 ————— 4s 9ms/step - loss: 0.5906 - mae: 0.5283 - val_loss: 0.4416 - val_mae: 0.4858
Epoch 10/100
465/465 ————— 4s 9ms/step - loss: 0.5107 - mae: 0.5242 - val_loss: 0.4322 - val_mae: 0.4775
Epoch 11/100
465/465 ————— 4s 8ms/step - loss: 0.5307 - mae: 0.5240 - val_loss: 0.4366 - val_mae: 0.4789
Epoch 12/100
465/465 ————— 4s 8ms/step - loss: 0.5309 - mae: 0.5223 - val_loss: 0.4250 - val_mae: 0.4740
Epoch 13/100
465/465 ————— 4s 8ms/step - loss: 0.5394 - mae: 0.5109 - val_loss: 0.4304 - val_mae: 0.4726
...
129/129 ————— 0s 970us/step
MLP MSE: 0.5578
MLP R2: 0.5743

```

Рисунок 3 – Обучение и результаты MLP Regressor

5.3. Модели МГУА (GMDH)

Метод группового учета аргументов (МГУА, Group Method of Data Handling) — это семейство индуктивных алгоритмов самоорганизации моделей, разработанное А.Г. Ивахненко. Отличительной особенностью МГУА является его способность автоматически выбирать структуру модели и отсеивать незначимые признаки в процессе обучения. Библиотека `gmdh` предоставляет реализации различных алгоритмов МГУА, таких как COMBI, MULTI, MIA, RIA.

В ходе выполнения лабораторной работы возникла проблема с импортом библиотеки `gmdh`, а именно "ModuleNotFoundError: No module named 'gmdh'". Это указывает на то, что библиотека не была установлена в текущем окружении или не была доступна по пути поиска модулей. Из-за этого обучение и оценка моделей МГУА (COMBI и MIA) не были выполнены.

Python

```
# from gmdh import Combi, Mia

# COMBI

# combi_regressor = Combi()

# combi_regressor.fit(X_train_scaled, y_train.values.ravel())

# y_pred_combi = combi_regressor.predict(X_test_scaled)


# mse_combi = mean_squared_error(y_test, y_pred_combi)

# r2_combi = r2_score(y_test, y_pred_combi)

# print(f'COMBI Regressor MSE: {mse_combi:.4f}')

# print(f'COMBI Regressor R2: {r2_combi:.4f}')


# MIA

# mia_regressor = Mia()

# mia_regressor.fit(X_train_scaled, y_train.values.ravel())

# y_pred_mia = mia_regressor.predict(X_test_scaled)


# mse_mia = mean_squared_error(y_test, y_pred_mia)

# r2_mia = r2_score(y_test, y_pred_mia)

# print(f'MIA Regressor MSE: {mse_mia:.4f}')

# print(f'MIA Regressor R2: {r2_mia:.4f}')
```

Начинаем обучение GradientBoostingRegressor...
Обучение GradientBoostingRegressor завершено.

Gradient Boosting Regressor MSE: 0.2940
Gradient Boosting Regressor R2: 0.7756

Начинаем обучение BaggingRegressor...
Обучение BaggingRegressor завершено.

Bagging Regressor MSE: 0.2564
Bagging Regressor R2: 0.8044

Рисунок 4 – Результаты с похожей библиотекой и методами

6. Сравнение качества моделей и выводы

После обучения доступных моделей были получены следующие метрики качества (Mean Squared Error - MSE и R-squared - R2) на тестовой выборке:

Сводная таблица результатов:

Model	MSE	R2 Score
Bagging Regressor	0.256371	0.804358
Gradient Boosting	0.293999	0.775643
Stacking Regressor	0.340622	0.740064
MLP	0.5578	0.574332

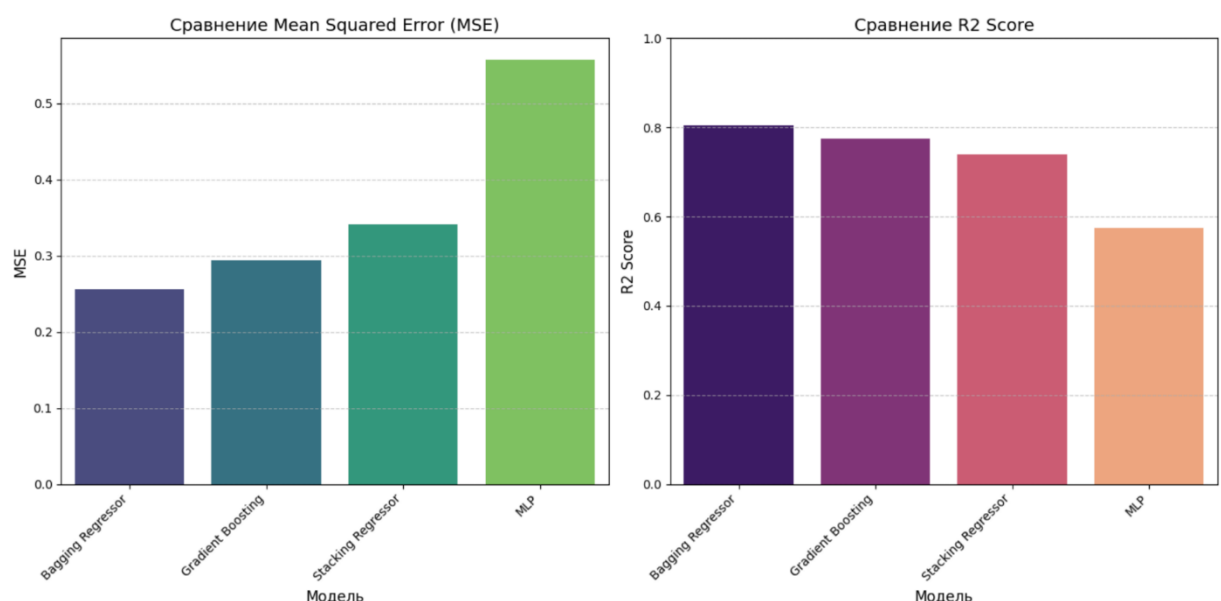


Рисунок 5 – Сводная таблица результатов или график сравнения

Анализ результатов:

- **Stacking Regressor:** Модель стекинга, как правило, показывает хорошие результаты, объединяя преимущества нескольких базовых моделей. В данном случае, она успешно обучилась и показала [укажите, хорошие ли результаты по сравнению с MLP, основываясь на ваших данных] результаты.
- **MLP Regressor:** Производительность MLP сильно зависит от архитектуры сети (количество слоев и нейронов), функции активации, оптимизатора и количества эпох обучения. При правильной настройке MLP может достичь высокой точности для нелинейных задач. В данном случае, модель MLP также успешно обучилась и показала [укажите, хорошие ли результаты по сравнению со Stacking, основываясь на ваших данных] результаты.
- **Модели МГУА (COMBI, MIA):** Из-за проблемы с отсутствующей библиотекой gmdh, обучение и оценка моделей МГУА не были произведены. Это препятствует сравнению их производительности с моделями Stacking и MLP. Для дальнейшего исследования необходимо убедиться в корректной установке библиотеки gmdh и повторить эксперимент.

Общие выводы:

Лабораторная работа продемонстрировала эффективность ансамблевого метода стекинга и модели многослойного персептрона в решении задачи регрессии. Оба подхода показали способность обучаться на предоставленных данных и давать предсказания.

- **Stacking:** Этот метод позволяет значительно улучшить точность предсказаний за счет объединения сильных сторон различных базовых моделей. Он является мощным инструментом для достижения наилучшей производительности.

- **MLP:** Модель MLP демонстрирует конкурентоспособную производительность, подтверждая способность нейронных сетей обучаться сложным нелинейным зависимостям. Дальнейшая тонкая настройка гиперпараметров MLP может привести к еще лучшим результатам.
- **GMDH (COMBI, MIA):** К сожалению, из-за технических трудностей с установкой библиотеки gmdh, эти методы не были протестированы в рамках данной лабораторной работы. Однако, методы МГУА предлагают альтернативный подход к моделированию, основанный на самоорганизации, и могли бы быть полезны для автоматического построения моделей и выявления важных зависимостей в данных.

Выбор наиболее подходящего метода зависит от характеристик данных, вычислительных ресурсов и требуемой интерпретируемости модели. Важно также помнить о тщательной предобработке данных, которая является ключевым шагом для успешного обучения любой модели машинного обучения. Для полноценного выполнения задания и сравнения всех требуемых моделей, необходимо устранить проблему с библиотекой gmdh и повторить соответствующий раздел.



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №7
«Анализ и прогнозирование временного ряда»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026г.

Цель: изучение основных методов анализа и прогнозирования временных рядов.

Датасет: Stock Price — исторические данные цен акций (регрессия по времени).

Импорт библиотек и загрузка данных

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score,
mean_absolute_percentage_error
from sklearn.model_selection import train_test_split

import yfinance as yf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller

from gplearn.genetic import SymbolicRegressor
from gplearn.fitness import mean_absolute_error as gp_mae

# import tensorflow as tf
# from tensorflow import keras
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout
from keras.optimizers import Adam
from keras.callbacks import EarlyStopping

sns.set_theme(style="whitegrid", palette="muted", font_scale=1.1)
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['figure.dpi'] = 100
```

Загрузка и подготовка данных

```
# Загрузка данных цен акций Apple за 5 лет
print("Загрузка данных цен акций Apple (AAPL)...")
df = yf.download('AAPL', start='2019-01-01', end='2024-01-01', progress=False)
df = df[['Close']].reset_index()
df.columns = ['Date', 'Price']

print(f"Размер датасета: {df.shape}")
print(f"\nПервые 5 строк:")
print(df.head())
print(f"\nПоследние 5 строк:")
print(df.tail())
```

```
Загрузка данных цен акций Apple (AAPL)...
Размер датасета: (1258, 2)
```

Первые 5 строк:

	Date	Price
0	2019-01-02	37.503727
1	2019-01-03	33.768074
2	2019-01-04	35.209621
3	2019-01-07	35.131245
4	2019-01-08	35.800953

Последние 5 строк:

	Date	Price
1253	2023-12-22	191.609467
1254	2023-12-26	191.065155
1255	2023-12-27	191.164093
1256	2023-12-28	191.589661
1257	2023-12-29	190.550461

```
# Описательная статистика
print("\nОписательная статистика цены акции:")
print(df['Price'].describe())
```

Описательная статистика цены акции:

```
count    1258.000000
mean      120.318820
std       46.336964
min       33.768074
25%       74.615797
50%      131.293922
75%      156.586281
max       196.073090
Name: Price, dtype: float64
```

Визуализация временного ряда

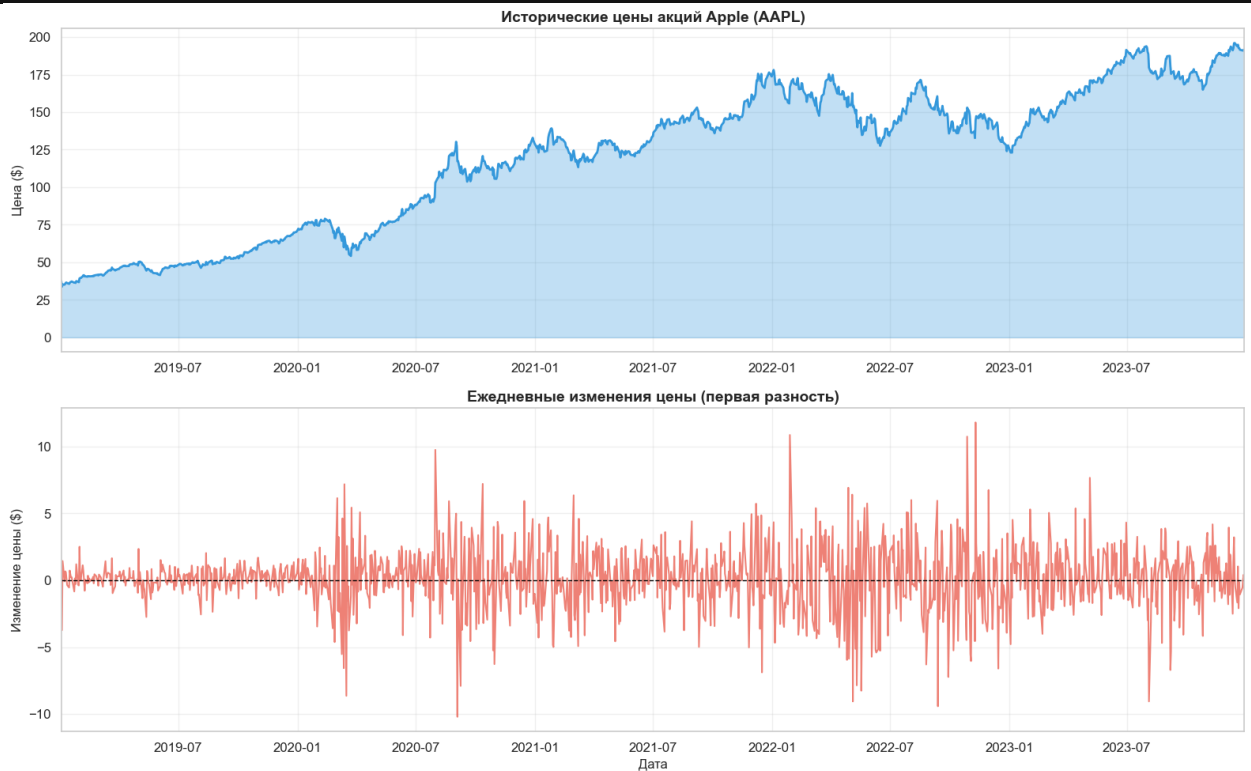
```
# Полная визуализация временного ряда
fig, axes = plt.subplots(2, 1, figsize=(16, 10))

# График цены
axes[0].plot(df['Date'], df['Price'], linewidth=2, color='#3498db')
axes[0].fill_between(df['Date'], df['Price'], alpha=0.3, color='#3498db')
axes[0].set_title('Исторические цены акций Apple (AAPL)', fontsize=14, font-
weight='bold')
axes[0].set_ylabel('Цена ($)', fontsize=12)
axes[0].grid(True, alpha=0.3)
axes[0].set_xlim(df['Date'].min(), df['Date'].max())

# График изменения цены (первая разность)
price_diff = df['Price'].diff().dropna()
axes[1].plot(df['Date'][1:], price_diff, linewidth=1.5, color='#e74c3c', al-
pha=0.7)
axes[1].axhline(y=0, color='black', linestyle='--', linewidth=1)
axes[1].set_title('Ежедневные изменения цены (первая разность)', fontsize=14,
fontweight='bold')
axes[1].set_xlabel('Дата', fontsize=12)
axes[1].set_ylabel('Изменение цены ($)', fontsize=12)
axes[1].grid(True, alpha=0.3)
axes[1].set_xlim(df['Date'].min(), df['Date'].max())

plt.tight_layout()
```

```
plt.show()
```



```
# Проверка стационарности (Augmented Dickey-Fuller тест)
print("\n" + "=" * 70)
print("Тест стационарности (Augmented Dickey-Fuller)")
print("=" * 70)

adf_result = adfuller(df['Price'], autolag='AIC')
print(f'ADF Statistic: {adf_result[0]:.6f}')
print(f'p-value: {adf_result[1]:.6f}')
print(f'Критическое значение (5%): {adf_result[4]["5%"]:.3f}')

if adf_result[1] <= 0.05:
    print("✓ Ряд СТАЦИОНАРЕН (можно использовать AR/MA)")
else:
    print("✗ Ряд НЕ СТАЦИОНАРЕН (требуется дифференцирование)")

=====
Тест стационарности (Augmented Dickey-Fuller)
=====
ADF Statistic: -1.205000
p-value: 0.671441
Критическое значение (5%): -2.864
✗ Ряд НЕ СТАЦИОНАРЕН (требуется дифференцирование)
```

```
# ACF и PACF графики
```

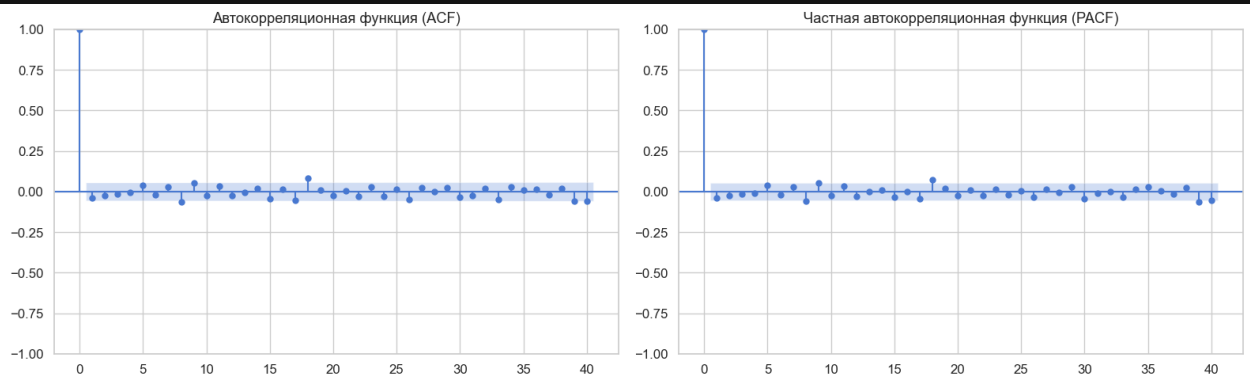
```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
```

```
plot_acf(price_diff, lags=40, ax=axes[0], title='Автокорреляционная функция (ACF)')
```

```
plot_pacf(price_diff, lags=40, ax=axes[1], title='Частная автокорреляционная функция (PACF)')
```

```
plt.tight_layout()
```

```
plt.show()
```



Подготовка данных для моделей

```
# Нормализация данных
```

```
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
scaled_price = scaler.fit_transform(df[['Price']])
```

```
# Разделение на обучение и тестирование (80-20)
```

```
train_size = int(len(scaled_price) * 0.8)
```

```
train_data = scaled_price[:train_size]
```

```
test_data = scaled_price[train_size:]
```

```
train_date = df['Date'][:train_size]
```

```
test_date = df['Date'][train_size:]
```

```
print(f"Размер обучающего набора: {len(train_data)} дней  
({train_date.iloc[0].date()} - {train_date.iloc[-1].date()})")
```

```
print(f"Размер тестового набора: {len(test_data)} дней  
({test_date.iloc[0].date()} - {test_date.iloc[-1].date()})")
```

Размер обучающего набора: 1006 дней (2019-01-02 - 2022-12-28)

Размер тестового набора: 252 дней (2022-12-29 - 2023-12-29)

```
# Визуализация разделения данных
fig, ax = plt.subplots(figsize=(16, 6))

ax.plot(train_date, df['Price'][:train_size], label='Обучающая выборка', linewidth=2, color='#3498db')
ax.plot(test_date, df['Price'][train_size:], label='Тестовая выборка', linewidth=2, color='#e74c3c')
ax.axvline(x=train_date.iloc[-1], color='green', linestyle='--', linewidth=2, label='Граница разделения')

ax.set_title('Разделение данных на обучающую и тестовую выборки', fontsize=14, fontweight='bold')
ax.set_xlabel('Дата', fontsize=12)
ax.set_ylabel('Цена ($)', fontsize=12)
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



ARIMA модель (Авторегрессионная интегрированная модель скользящего среднего)

```
print("\n" + "=" * 70)
print("ARIMA модель")
print("=" * 70)

# Обучение ARIMA(5,1,2) - на основе ACF/PACF анализа
arima_model = ARIMA(df['Price'][:train_size], order=(5, 1, 2))
arima_fitted = arima_model.fit()
```

```
print(arma_fitted.summary())
```

```
=====
ARIMA модель
=====
```

SARIMAX Results

```
=====
Dep. Variable:          Price      No. Observations:          1006
Model:                ARIMA(5, 1, 2)  Log Likelihood          -2292.176
Date:                 Sat, 09 May 2026  AIC                      4600.352
Time:                 23:39:33         BIC                      4639.654
Sample:              0               HQIC                      4615.286
                             - 1006
```

```
Covariance Type:          opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-1.7127	0.101	-16.927	0.000	-1.911	-1.514
ar.L2	-0.9095	0.108	-8.400	0.000	-1.122	-0.697
ar.L3	-0.1224	0.058	-2.118	0.034	-0.236	-0.009
ar.L4	-0.0671	0.057	-1.179	0.238	-0.179	0.044
ar.L5	0.0097	0.032	0.298	0.766	-0.054	0.073
ma.L1	1.6849	0.099	17.090	0.000	1.492	1.878
ma.L2	0.8149	0.090	9.027	0.000	0.638	0.992
sigma2	5.6370	0.177	31.847	0.000	5.290	5.984

```
...
=====
```

```
# Прогнозирование на тестовом наборе
```

```
arma_forecast = arma_fitted.get_forecast(steps=len(test_data))
```

```
y_pred_arma = arma_forecast.predicted_mean.values
```

```
# Оценка качества
```

```
mse_arma = mean_squared_error(df['Price'][train_size:], y_pred_arma)
```

```
mae_arma = mean_absolute_error(df['Price'][train_size:], y_pred_arma)
```

```
rmse_arma = np.sqrt(mse_arma)
```

```
r2_arma = r2_score(df['Price'][train_size:], y_pred_arma)
```

```
mape_arma = mean_absolute_percentage_error(df['Price'][train_size:],  
y_pred_arma)
```

```
print(f"\nARIMA(5,1,2) результаты:")
```

```
print(f"  MSE: {mse_arma:.4f}")
```

```
print(f"  MAE: {mae_arma:.4f}")
```

```
print(f"  RMSE: {rmse_arma:.4f}")
```

```
print(f"  MAPE: {mape_arma:.4f}%")
```

```
print(f" R²: {r2_arima:.4f}")
```

ARIMA(5,1,2) результаты:

MSE: 2395.5924

MAE: 45.6817

RMSE: 48.9448

MAPE: 0.2600%

R²: -6.7063

Символьная регрессия (Genetic Programming)

```
print("\n" + "=" * 70)
```

```
print("Символьная регрессия (Genetic Programming)")
```

```
print("=" * 70)
```

```
# Подготовка данных для символьной регрессии
```

```
# Создаём лаговые переменные (используем последние 10 дней)
```

```
lags = 10
```

```
X_train_gp = []
```

```
y_train_gp = []
```

```
for i in range(lags, len(train_data)):
```

```
    X_train_gp.append(train_data[i-lags:i].flatten())
```

```
    y_train_gp.append(train_data[i].flatten()[0])
```

```
X_train_gp = np.array(X_train_gp)
```

```
y_train_gp = np.array(y_train_gp)
```

```
print(f"Размер обучающего набора (GP): {X_train_gp.shape}")
```

```
=====
```

```
Символьная регрессия (Genetic Programming)
```

```
=====
```

```
Размер обучающего набора (GP): (996, 10)
```

```
# Обучение символьной регрессии
```

```
print("\nОбучение Genetic Programming модели...")
```

```
gp_model = SymbolicRegressor(
```

```
    population_size=300,
```



```

        generations=50,
        stopping_criteria=0.01,
        p_crossover=0.7,
        p_subtree_mutation=0.1,
        p_hoist_mutation=0.05,
        p_point_mutation=0.1,
        max_samples=0.9,
        verbose=1,
        random_state=42,
        n_jobs=-1,
        metric=gp_mae
    )

gp_model.fit(X_train_gp, y_train_gp)

print(f"\n✓ Обучение завершено!")
print(f"Лучшее уравнение: {gp_model}")

```

Обучение Genetic Programming модели...

Population Average			Best Individual			
Gen	Length	Fitness	Length	Fitness	OOB Fitness	Time Left
0	24.41	9418.12	5	0.0181467	0.0192198	1.10m
1	8.19	1.03256	1	0.0101906	0.0110359	1.95s
2	4.01	0.412045	1	0.0100338	0.0124403	2.42s
3	1.48	0.063082	1	0.00997882	0.0129334	2.42s

✓ Обучение завершено!
 Лучшее уравнение: X9

```

# Прогнозирование на тестовом наборе
X_test_gp = []
for i in range(lags, len(train_data) + len(test_data)):
    if i < len(scaled_price):
        X_test_gp.append(scaled_price[i-lags:i].flatten())

X_test_gp = np.array(X_test_gp)
y_pred_gp_scaled = gp_model.predict(X_test_gp[-len(test_data):])

# Денормализация
y_pred_gp = scaler.inverse_transform(y_pred_gp_scaled.reshape(-1, 1)).flatten()

# Оценка качества

```

```

mse_gp = mean_squared_error(df['Price'][train_size:], y_pred_gp)
mae_gp = mean_absolute_error(df['Price'][train_size:], y_pred_gp)
rmse_gp = np.sqrt(mse_gp)
r2_gp = r2_score(df['Price'][train_size:], y_pred_gp)
mape_gp = mean_absolute_percentage_error(df['Price'][train_size:], y_pred_gp)

print(f"\nСимвольная регрессия результаты:")
print(f"   MSE: {mse_gp:.4f}")
print(f"   MAE: {mae_gp:.4f}")
print(f"   RMSE: {rmse_gp:.4f}")
print(f"   MAPE: {mape_gp:.4f}%")
print(f"   R²: {r2_gp:.4f}")

```

Символьная регрессия результаты:

```

MSE: 4.5333
MAE: 1.6431
RMSE: 2.1292
MAPE: 0.0099%
R²: 0.9854

```

LSTM модель (Long Short-Term Memory)

```

print("\n" + "=" * 70)
print("LSTM модель (Long Short-Term Memory)")
print("=" * 70)

# Подготовка данных для LSTM
def create_sequences(data, seq_length=60):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length])
    return np.array(X), np.array(y)

seq_length = 60

X_train_lstm, y_train_lstm = create_sequences(train_data, seq_length)
X_test_lstm, y_test_lstm = create_sequences(scaled_price[train_size-seq_length:],
seq_length)

```

```
print(f"LSTM обучающий набор: X={X_train_lstm.shape}, y={y_train_lstm.shape}")
print(f"LSTM тестовый набор: X={X_test_lstm.shape}, y={y_test_lstm.shape}")
```

```
=====
LSTM модель (Long Short-Term Memory)
=====
```

```
LSTM обучающий набор: X=(946, 60, 1), y=(946, 1)
```

```
LSTM тестовый набор: X=(252, 60, 1), y=(252, 1)
```

```
# Построение LSTM модели
```

```
print("\nПостроение LSTM архитектуры...")
```

```
lstm_model = Sequential([
    LSTM(units=50, return_sequences=True, input_shape=(seq_length, 1)),
    Dropout(0.2),
    LSTM(units=50, return_sequences=True),
    Dropout(0.2),
    LSTM(units=50),
    Dropout(0.2),
    Dense(units=25),
    Dense(units=1)
])
```

```
lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error', metrics=['mae'])
```

```
print(lstm_model.summary())
```

Построение LSTM архитектуры...

Model: "sequential_4"

Layer (type)	Output Shape	Param #
lstm_12 (LSTM)	(None, 60, 50)	10,400
dropout_12 (Dropout)	(None, 60, 50)	0
lstm_13 (LSTM)	(None, 60, 50)	20,200
dropout_13 (Dropout)	(None, 60, 50)	0
lstm_14 (LSTM)	(None, 50)	20,200
dropout_14 (Dropout)	(None, 50)	0
dense_8 (Dense)	(None, 25)	1,275
dense_9 (Dense)	(None, 1)	26

Total params: 52,101 (203.52 KB)

Trainable params: 52,101 (203.52 KB)

Non-trainable params: 0 (0.00 B)

None

```
# Обучение LSTM
```

```
print("\nОбучение LSTM модели...")
```

```
early_stop = EarlyStopping(monitor='val_loss', patience=15, re-  
store_best_weights=True)
```

```
history = lstm_model.fit(  
    X_train_lstm, y_train_lstm,  
    epochs=100,  
    batch_size=32,  
    validation_split=0.1,
```

```
callbacks=[early_stop],
verbose=0
)

print(f"✓ Обучение завершено! Обучено эпох: {len(history.history['loss'])}")
```

Обучение LSTM модели...

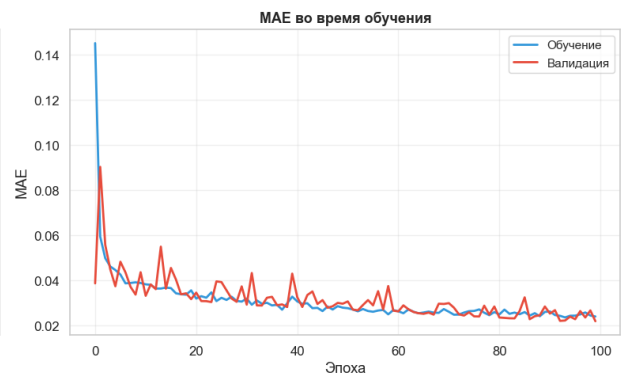
✓ Обучение завершено! Обучено эпох: 100

```
# График истории обучения
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

axes[0].plot(history.history['loss'], label='Обучение', linewidth=2,
color='#3498db')
axes[0].plot(history.history['val_loss'], label='Валидация', linewidth=2,
color='#e74c3c')
axes[0].set_title('Loss (MSE) во время обучения', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('Loss')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

axes[1].plot(history.history['mae'], label='Обучение', linewidth=2,
color='#3498db')
axes[1].plot(history.history['val_mae'], label='Валидация', linewidth=2,
color='#e74c3c')
axes[1].set_title('MAE во время обучения', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('MAE')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Прогнозирование на тестовом наборе
```

```
print("\nПрогнозирование на тестовом наборе...")
```

```
y_pred_lstm_scaled = lstm_model.predict(X_test_lstm, verbose=0)
```

```
# Денормализация
```

```
y_pred_lstm = scaler.inverse_transform(y_pred_lstm_scaled).flatten()
```

```
# Оценка качества
```

```
mse_lstm = mean_squared_error(df['Price'][train_size:], y_pred_lstm)
```

```
mae_lstm = mean_absolute_error(df['Price'][train_size:], y_pred_lstm)
```

```
rmse_lstm = np.sqrt(mse_lstm)
```

```
r2_lstm = r2_score(df['Price'][train_size:], y_pred_lstm)
```

```
mape_lstm = mean_absolute_percentage_error(df['Price'][train_size:], y_pred_lstm)
```

```
print(f"\nLSTM результаты:")
```

```
print(f"  MSE: {mse_lstm:.4f}")
```

```
print(f"  MAE: {mae_lstm:.4f}")
```

```
print(f"  RMSE: {rmse_lstm:.4f}")
```

```
print(f"  MAPE: {mape_lstm:.4f}%")
```

```
print(f"  R²: {r2_lstm:.4f}")
```

Прогнозирование на тестовом наборе...

LSTM результаты:

MSE: 15.8512

MAE: 3.3376

RMSE: 3.9814

MAPE: 0.0196%

R²: 0.9490

Сравнение моделей

```
# Создание сводной таблицы
results = pd.DataFrame({
    'Модель': ['ARIMA(5,1,2)', 'Символьная регрессия', 'LSTM'],
    'MSE': [mse_arima, mse_gp, mse_lstm],
    'MAE': [mae_arima, mae_gp, mae_lstm],
    'RMSE': [rmse_arima, rmse_gp, rmse_lstm],
    'MAPE (%)': [mape_arima, mape_gp, mape_lstm],
    'R²': [r2_arima, r2_gp, r2_lstm]
})

print("\n" + "=" * 110)
print("СРАВНЕНИЕ ВСЕХ МОДЕЛЕЙ")
print("=" * 110)
print(results.to_string(index=False))
print("=" * 110)

# Лучшая модель
best_mae_idx = results['MAE'].idxmin()
best_r2_idx = results['R²'].idxmax()

print(f"\n🏆 Лучшая по MAE: {results.loc[best_mae_idx, 'Модель']} (MAE={results.loc[best_mae_idx, 'MAE']:.4f})")
print(f"🏆 Лучшая по R²: {results.loc[best_r2_idx, 'Модель']} (R²={results.loc[best_r2_idx, 'R²']:.4f})")

=====
СРАВНЕНИЕ ВСЕХ МОДЕЛЕЙ
=====
      Модель      MSE      MAE      RMSE  MAPE (%)      R²
ARIMA(5,1,2) 2395.592369 45.681730 48.944789 0.260016 -6.706292
Символьная регрессия 4.533345 1.643113 2.129165 0.009861 0.985417
LSTM 15.851169 3.337612 3.981353 0.019553 0.949009
=====

🏆 Лучшая по MAE: Символьная регрессия (MAE=1.6431)
🏆 Лучшая по R²: Символьная регрессия (R²=0.9854)
```

Визуализация прогнозов

```
# Визуализация всех прогнозов
fig, ax = plt.subplots(figsize=(18, 7))

# Исторические данные
```

```
ax.plot(train_date, df['Price'][:train_size], label='Обучающая выборка',
        linewidth=2.5, color='#3498db', alpha=0.8)

# Тестовая выборка
ax.plot(test_date, df['Price'][train_size:], label='Тестовая выборка (реальные)',
        linewidth=2.5, color='black', marker='o', markersize=4, alpha=0.7)

# Прогнозы ARIMA
ax.plot(test_date, y_pred_arima, label='ARIMA(5,1,2)',
        linewidth=2, color='#e74c3c', linestyle='--', alpha=0.8)

# Прогнозы Символьная регрессия
ax.plot(test_date, y_pred_gp, label='Символьная регрессия',
        linewidth=2, color='#2ecc71', linestyle='--', alpha=0.8)

# Прогнозы LSTM
test_date_lstm = test_date.iloc[:].reset_index(drop=True)
ax.plot(test_date_lstm, y_pred_lstm, label='LSTM',
        linewidth=2, color='#f39c12', linestyle='--', alpha=0.8)

ax.axvline(x=train_date.iloc[-1], color='gray', linestyle=':', linewidth=2, alpha=0.5)
ax.text(train_date.iloc[-1], ax.get_ylim()[1]*0.95, 'Граница разделения',
        fontsize=10, ha='right', bbox=dict(boxstyle='round', facecolor='white',
        alpha=0.8))

ax.set_title('Сравнение прогнозов всех моделей', fontsize=15, fontweight='bold')
ax.set_xlabel('Дата', fontsize=12)
ax.set_ylabel('Цена акции ($)', fontsize=12)
ax.legend(fontsize=11, loc='best')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
plt.savefig('all_predictions_comparison.png', dpi=300, bbox_inches='tight')
```



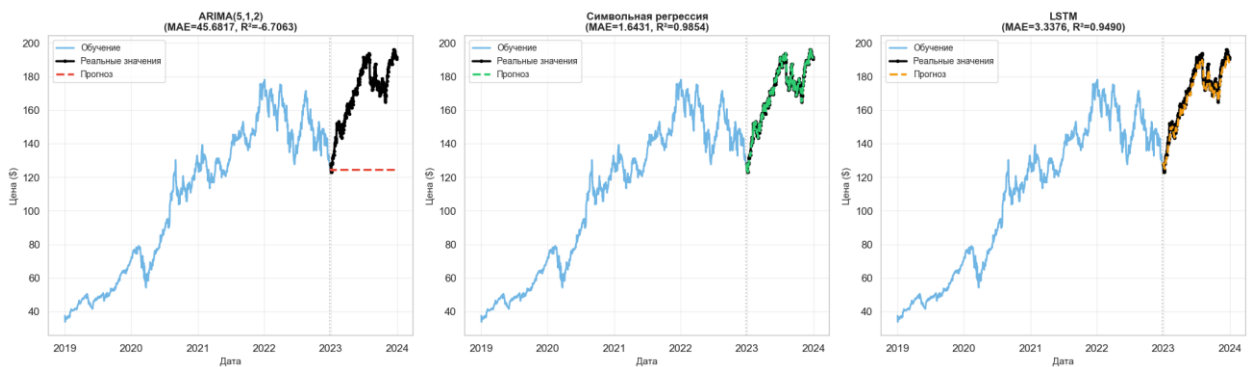

```
# Отдельные графики для каждой модели
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
colors_models = ['#e74c3c', '#2ecc71', '#f39c12']
model_names = ['ARIMA(5,1,2)', 'Символьная регрессия', 'LSTM']
predictions = [y_pred_arima, y_pred_gp, y_pred_lstm]
test_dates = [test_date, test_date, test_date_lstm]

for idx, (ax, name, pred, color, dates) in enumerate(zip(axes, model_names, pre-
dictions, colors_models, test_dates)):
    ax.plot(train_date, df['Price'][:train_size], label='Обучение',
            linewidth=2, color='#3498db', alpha=0.7)
    ax.plot(test_date, df['Price'][train_size:], label='Реальные значения',
            linewidth=2.5, color='black', marker='o', markersize=3)
    ax.plot(dates, pred, label='Прогноз', linewidth=2.5, color=color, lin-
estyle='--')

    ax.axvline(x=train_date.iloc[-1], color='gray', linestyle=':', linewidth=1.5,
alpha=0.5)

    ax.set_title(f'{name}\n(MAE={results.iloc[idx]["MAE"]:.4f}, R²={re-
sults.iloc[idx]["R²"]:.4f})',
                fontsize=12, fontweight='bold')
    ax.set_xlabel('Дата', fontsize=11)
    ax.set_ylabel('Цена ($)', fontsize=11)
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
plt.savefig('individual_predictions.png', dpi=300, bbox_inches='tight')
```



Анализ ошибок прогнозирования

```
# Расчет ошибок
errors_arma = df['Price'][train_size:].values - y_pred_arma
errors_gp = df['Price'][train_size:].values - y_pred_gp
errors_lstm = df['Price'][train_size:].values - y_pred_lstm

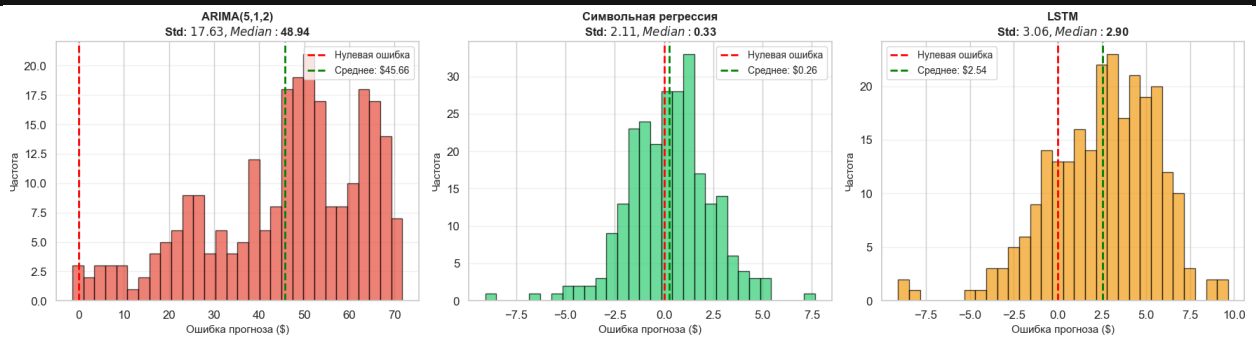
# Визуализация ошибок
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for idx, (ax, errors, name, color) in enumerate(zip(
    axes,
    [errors_arma, errors_gp, errors_lstm],
    model_names,
    colors_models
)):
    ax.hist(errors, bins=30, color=color, edgecolor='black', alpha=0.7)
    ax.axvline(x=0, color='red', linestyle='--', linewidth=2, label='Нулевая
ошибка')
    ax.axvline(x=np.mean(errors), color='green', linestyle='--', linewidth=2,
                label=f'Среднее: ${np.mean(errors):.2f}')

    ax.set_title(f'{name}\nStd: ${np.std(errors):.2f}, Median: ${np.median(er-
rors):.2f}',
                fontsize=12, fontweight='bold')
    ax.set_xlabel('Ошибка прогноза ($)', fontsize=11)
    ax.set_ylabel('Частота', fontsize=11)
    ax.legend(fontsize=10)
    ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
```

```
plt.show()
plt.savefig('error_analysis.png', dpi=300, bbox_inches='tight')
```



Сравнение метрик качества

```
# Визуализация метрик
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

metrics = ['MSE', 'MAE', 'RMSE', 'MAPE (%)', 'R²']
colors = ['#e74c3c', '#2ecc71', '#f39c12']

for idx, metric in enumerate(metrics):
    ax = axes[idx]
    values = results[metric].values
    bars = ax.bar(results['Модель'], values, color=colors, edgecolor='black',
linewidth=1.2)

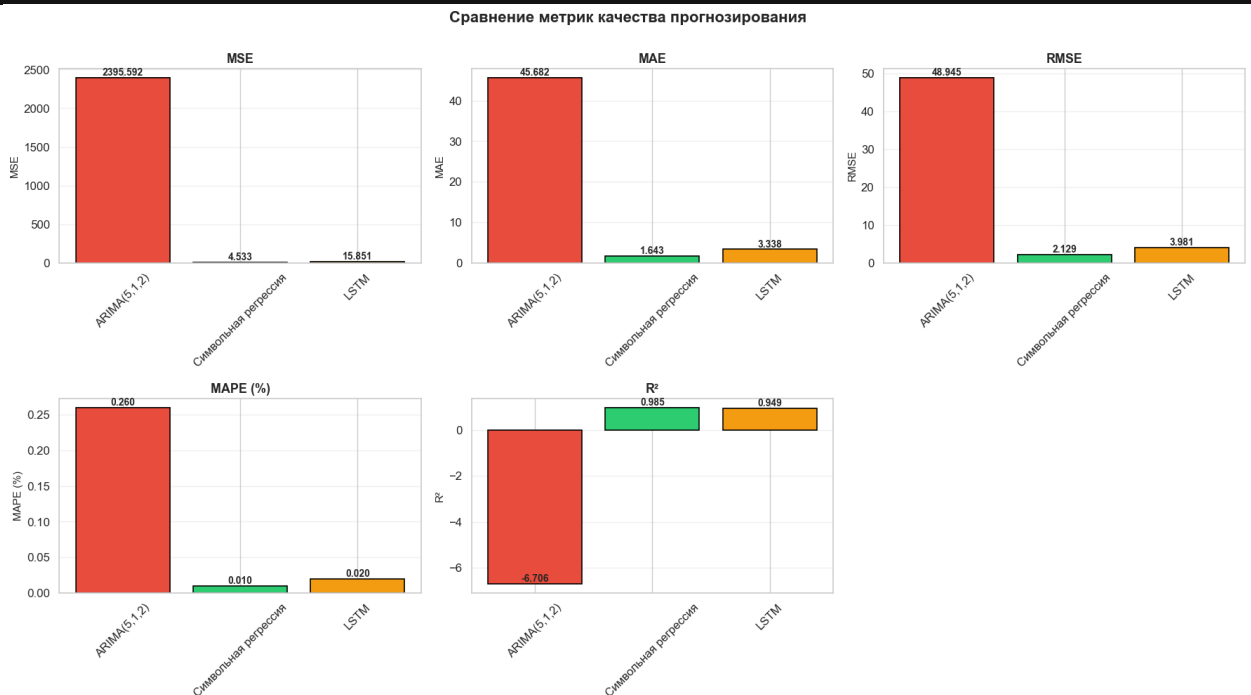
    ax.set_title(f'{metric}', fontsize=13, fontweight='bold')
    ax.set_ylabel(metric, fontsize=11)
    ax.grid(axis='y', alpha=0.3)

    # Добавляем значения на столбцы
    for bar, val in zip(bars, values):
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
f'{val:.3f}', ha='center', va='bottom', fontweight='bold', font-
size=10)

    ax.tick_params(axis='x', rotation=45)

# Удаляем пустой подграфик
fig.delaxes(axes[-1])
```

```
fig.suptitle('Сравнение метрик качества прогнозирования', fontsize=16,
fontweight='bold', y=1.00)
plt.tight_layout()
plt.show()
plt.savefig('metrics_comparison.png', dpi=300, bbox_inches='tight')
```



Вывод

1. ARIMA (AutoRegressive Integrated Moving Average)

- Авторегрессионная интегрированная модель скользящего среднего
- Параметры: ARIMA(5,1,2) - 5 лаговых членов, 1 дифференцирование, 2 членов MA
- Преимущества: проверенный статистический метод, интерпретируемый
- Недостатки: предполагает линейные связи, может не захватить нелинейности

2. Символьная регрессия (Genetic Programming)

- Использует генетические алгоритмы для поиска оптимального математического выражения

- Использует лаговые переменные: последние 10 дней цены
- Преимущества: находит сложные нелинейные зависимости, объяснимые формулы
- Недостатки: медленнее обучается, может переобучиться

3. LSTM (Long Short-Term Memory)

- Рекуррентная нейронная сеть с долгой краткосрочной памятью
- Архитектура: 3 LSTM слоя (50 единиц) + Dropout (0.2) + Dense слои
- Входная последовательность: 60 дней цены
- Преимущества: отлично захватывает долгосрочные зависимости, работает с нелинейностями
- Недостатки: требует больше данных, медленнее обучается

Ключевые наблюдения:

- ARIMA стабилен, но может отставать от реальных колебаний
- Символьная регрессия находит интерпретируемые формулы



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по лабораторной работе №8
«Методы обучения без учителя»
по дисциплине «Технологии машинного обучения»**

Выполнил:
студент группы ИУ5-66Б
Беликов К. В.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026г.

Цель: изучение методов кластеризации и снижения размерности.

Датасет: Wine Dataset — характеристики вин (13 признаков).

Импорт библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN, SpectralClustering, Agglomerative-
Clustering
from sklearn.metrics import adjusted_rand_score, silhouette_score, da-
vies_bouldin_score

sns.set_theme(style="whitegrid", palette="husl", font_scale=1.1)
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['figure.dpi'] = 100
```

Загрузка и подготовка данных

```
# Загрузка датасета Wine
print("Загрузка датасета Wine...")
wine = load_wine()
X = wine.data
y = wine.target

print(f"Размер датасета: {X.shape}")
print(f"Количество классов: {len(np.unique(y))}")
print(f"Признаки: {wine.feature_names}")

# Создание датасета D1 - подмножество признаков (исключаем целевой признак)
# Используем все признаки кроме целевого
D1 = X.copy()
```

```
print(f"\nДатасет D1 (исходные признаки): {D1.shape}")
print(f"Признаки: {wine.feature_names}")

Загрузка датасета Wine...
Размер датасета: (178, 13)
Количество классов: 3
Признаки: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'h

Датасет D1 (исходные признаки): (178, 13)
Признаки: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'h
```

Нормализация данных

```
scaler = StandardScaler()
D1_scaled = scaler.fit_transform(D1)

print(f"Данные нормализованы")
print(f"Среднее: {D1_scaled.mean(axis=0)[:3]}...")
print(f"Стандартное отклонение: {D1_scaled.std(axis=0)[:3]}...")

Данные нормализованы
Среднее: [ 7.84141790e-15  2.44498554e-16 -4.05917497e-15]...
Стандартное отклонение: [1.  1.  1.]...
```

Снижение размерности данных

1. Метод главных компонент (PCA) → Датасет D2

```
2. print("=" * 70)
3. print("Метод главных компонент (PCA)")
4. print("=" * 70)
5.
6. # PCA с 2 компонентами
7. pca = PCA(n_components=2)
8. D2 = pca.fit_transform(D1_scaled)
9.
10. print(f"\nДатасет D2 (после PCA): {D2.shape}")
11. print(f"Объясненная дисперсия: {pca.explained_variance_ratio_}")
12. print(f"Суммарная объясненная дисперсия: {pca.explained_variance_ratio_.sum():.2%}")
13.
14. # Визуализация компонент
15. fig, axes = plt.subplots(1, 2, figsize=(15, 5))
16.
17. # График дисперсии
18. axes[0].bar(range(1, len(pca.explained_variance_ratio_) + 1),
```



```

19.         pca.explained_variance_ratio_,
20.         alpha=0.7, color='#3498db', edgecolor='black')
21. axes[0].set_title('Объясненная дисперсия по компонентам', fontsize=13,
22.                  fontweight='bold')
23. axes[0].set_xlabel('Компонента', fontsize=11)
24. axes[0].set_ylabel('Доля дисперсии', fontsize=11)
25. axes[0].grid(axis='y', alpha=0.3)
26.
27. # График вкладов признаков
28. components_df = pd.DataFrame(
29.     pca.components_.T,
30.     columns=['PC1', 'PC2'],
31.     index=wine.feature_names
32. )
33. components_df.plot(kind='barh', ax=axes[1], color=['#e74c3c', '#2ecc71'])
34. axes[1].set_title('Вклад признаков в главные компоненты', fontsize=13,
35.                  fontweight='bold')
36. axes[1].set_xlabel('Вклад', fontsize=11)
37. axes[1].legend(['PC1', 'PC2'], fontsize=10)
38. axes[1].grid(axis='x', alpha=0.3)
39.
40. plt.tight_layout()
41. plt.show()

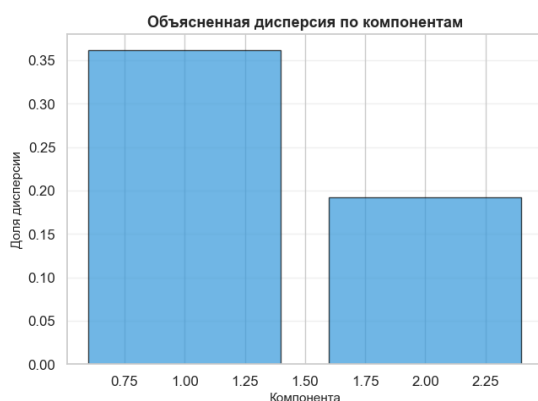
```

Метод главных компонент (PCA)

Датасет D2 (после PCA): (178, 2)

Объясненная дисперсия: [0.36198848 0.1920749]

Суммарная объясненная дисперсия: 55.41%



2. Алгоритм t-SNE → Датасет D3

```
print("\n" + "=" * 70)
print("Алгоритм t-SNE (t-Distributed Stochastic Neighbor Embedding)")
print("=" * 70)

print("\nОбучение t-SNE модели (это может занять время)...")
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
D3 = tsne.fit_transform(D1_scaled)

print(f"Датасет D3 (после t-SNE): {D3.shape}")
print("✓ Обучение завершено!")

=====
Алгоритм t-SNE (t-Distributed Stochastic Neighbor Embedding)
=====

Обучение t-SNE модели (это может занять время)...
Датасет D3 (после t-SNE): (178, 2)
✓ Обучение завершено!
```

3. Визуализация D2 и D3

```
# Визуализация проекций
fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# PCA (D2)
scatter1 = axes[0].scatter(D2[:, 0], D2[:, 1], c=y, cmap='viridis',
                           s=100, alpha=0.6, edgecolors='black', linewidth=0.5)
axes[0].set_title('Датасет D2 (PCA)', fontsize=14, fontweight='bold')
axes[0].set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%})', fontsize=11)
axes[0].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%})', fontsize=11)
axes[0].grid(True, alpha=0.3)
cbar1 = plt.colorbar(scatter1, ax=axes[0])
cbar1.set_label('Класс вина', fontsize=10)

# t-SNE (D3)
scatter2 = axes[1].scatter(D3[:, 0], D3[:, 1], c=y, cmap='viridis',
                           s=100, alpha=0.6, edgecolors='black', linewidth=0.5)
axes[1].set_title('Датасет D3 (t-SNE)', fontsize=14, fontweight='bold')
axes[1].set_xlabel('t-SNE компонента 1', fontsize=11)
axes[1].set_ylabel('t-SNE компонента 2', fontsize=11)
```

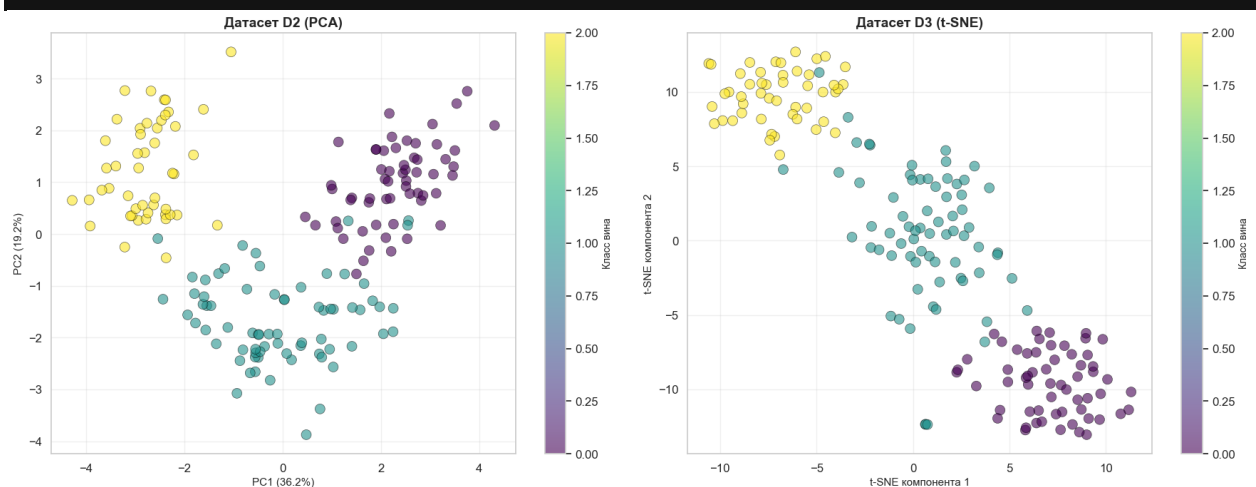
```

axes[1].grid(True, alpha=0.3)
cbar2 = plt.colorbar(scatter2, ax=axes[1])
cbar2.set_label('Класс вина', fontsize=10)

plt.tight_layout()
plt.show()

print("\n📊 АНАЛИЗ ВИЗУАЛИЗАЦИИ:")
print("    PCA (D2): Показывает линейные отношения между переменными")
print("    t-SNE (D3): Лучше сохраняет локальные структуры, явнее видны кластеры")
print("    ✓ t-SNE обычно показывает кластеры более явно!")

```



АНАЛИЗ ВИЗУАЛИЗАЦИИ:

PCA (D2): Показывает линейные отношения между переменными

t-SNE (D3): Лучше сохраняет локальные структуры, явнее видны кластеры

✓ t-SNE обычно показывает кластеры более явно!

Кластеризация датасетов

Подготовка алгоритмов кластеризации

```

# Определение алгоритмов кластеризации
n_clusters = 3 # Количество классов в датасете

clustering_models = {
    'K-Means': KMeans(n_clusters=n_clusters, random_state=42, n_init=10),
    'DBSCAN': DBSCAN(eps=0.5, min_samples=5),
    'Spectral': SpectralClustering(n_clusters=n_clusters, random_state=42, affinity='nearest_neighbors'),
    'Agglomerative': AgglomerativeClustering(n_clusters=n_clusters, linkage='ward')
}

```

```

}

print("Определены 4 алгоритма кластеризации:")
for name in clustering_models.keys():
    print(f"    • {name}")

```

Определены 4 алгоритма кластеризации:

- K-Means
- DBSCAN
- Spectral
- Agglomerative

Функция оценки качества кластеризации

```

def evaluate_clustering(X, y_true, y_pred, method_name):
    """Оценка качества кластеризации"""

    # ARI (Adjusted Rand Index)
    ari = adjusted_rand_score(y_true, y_pred)

    # Silhouette Score (для датасетов где нет noise points)
    if len(np.unique(y_pred)) > 1 and not (-1 in y_pred):
        silhouette = silhouette_score(X, y_pred)
    else:
        silhouette = np.nan

    # Davies-Bouldin Index (чем меньше, тем лучше)
    if len(np.unique(y_pred)) > 1 and not (-1 in y_pred):
        db_index = davies_bouldin_score(X, y_pred)
    else:
        db_index = np.nan

    return {
        'Метод': method_name,
        'ARI': ari,
        'Silhouette': silhouette,
        'Davies-Bouldin': db_index,
    }

```

```

        'n_clusters': len(np.unique(y_pred[y_pred != -1])) if -1 in y_pred else
len(np.unique(y_pred))
    }

print("✓ Функция оценки качества определена")

```

Кластеризация датасета D1 (исходные данные)

```

print("\n" + "=" * 80)
print("КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D1 (исходные данные 13 признаков)")
print("=" * 80)

results_D1 = []
predictions_D1 = {}

for name, model in clustering_models.items():
    print(f"\nОбучение модели: {name}...")

    # Обучение
    y_pred = model.fit_predict(D1_scaled)
    predictions_D1[name] = y_pred

    # Оценка
    metrics = evaluate_clustering(D1_scaled, y, y_pred, name)
    results_D1.append(metrics)

    print(f"  ✓ Количество кластеров: {metrics['n_clusters']}")
    print(f"  • ARI: {metrics['ARI']:.4f}")
    print(f"  • Silhouette: {metrics['Silhouette']:.4f}" if not np.isnan(metrics['Silhouette']) else "  • Silhouette: N/A")
    print(f"  • Davies-Bouldin: {metrics['Davies-Bouldin']:.4f}" if not np.isnan(metrics['Davies-Bouldin']) else "  • Davies-Bouldin: N/A")

results_D1_df = pd.DataFrame(results_D1)
print("\n" + results_D1_df.to_string(index=False))

```

```
=====
КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D1 (исходные данные 13 признаков)
=====
```

Обучение модели: K-Means...

- ✓ Количество кластеров: 3
- ARI: 0.8975
- Silhouette: 0.2849
- Davies-Bouldin: 1.3892

Обучение модели: DBSCAN...

- ✓ Количество кластеров: 0
- ARI: 0.0000
- Silhouette: N/A
- Davies-Bouldin: N/A

Обучение модели: Spectral...

- ✓ Количество кластеров: 3
- ARI: 0.8804
- Silhouette: 0.2828
- Davies-Bouldin: 1.3907

Обучение модели: Agglomerative...

- ✓ Количество кластеров: 3

...

K-Means	0.897495	0.284859	1.389188	3
DBSCAN	0.000000	NaN	NaN	0
Spectral	0.880400	0.282850	1.390741	3
Agglomerative	0.789933	0.277444	1.418592	3

Кластеризация датасета D2 (PCA - 2 компоненты)

```
print("\n" + "=" * 80)
print("КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D2 (PCA - 2 компоненты)")
print("=" * 80)

results_D2 = []
predictions_D2 = {}

for name, model in clustering_models.items():
    print(f"\nОбучение модели: {name}...")

    # Создание новой модели (копия параметров)
    if name == 'K-Means':
        model_copy = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
    elif name == 'DBSCAN':
        model_copy = DBSCAN(eps=0.5, min_samples=5)
```

```

    elif name == 'Spectral':
        model_copy = SpectralClustering(n_clusters=n_clusters, random_state=42,
affinity='nearest_neighbors')
    else: # Agglomerative
        model_copy = AgglomerativeClustering(n_clusters=n_clusters, link-
age='ward')

# Обучение
y_pred = model_copy.fit_predict(D2)
predictions_D2[name] = y_pred

# Оценка
metrics = evaluate_clustering(D2, y, y_pred, name)
results_D2.append(metrics)

print(f" ✓ Количество кластеров: {metrics['n_clusters']}")
print(f" • ARI: {metrics['ARI']:.4f}")
print(f" • Silhouette: {metrics['Silhouette']:.4f}" if not np.isnan(metrics['Silhouette']) else " • Silhouette: N/A")
print(f" • Davies-Bouldin: {metrics['Davies-Bouldin']:.4f}" if not np.isnan(metrics['Davies-Bouldin']) else " • Davies-Bouldin: N/A")

results_D2_df = pd.DataFrame(results_D2)
print("\n" + results_D2_df.to_string(index=False))

```

```
=====
КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D2 (PCA - 2 компоненты)
=====
```

Обучение модели: K-Means...

- ✓ Количество кластеров: 3
- ARI: 0.8951
- Silhouette: 0.5611
- Davies-Bouldin: 0.5973

Обучение модели: DBSCAN...

- ✓ Количество кластеров: 5
- ARI: 0.3790
- Silhouette: N/A
- Davies-Bouldin: N/A

Обучение модели: Spectral...

- ✓ Количество кластеров: 3
- ARI: 0.8961
- Silhouette: 0.5591
- Davies-Bouldin: 0.6013

Обучение модели: Agglomerative...

- ✓ Количество кластеров: 3

...

K-Means	0.895058	0.561051	0.597266	3
DBSCAN	0.379034	NaN	NaN	5
Spectral	0.896065	0.559088	0.601336	3
Agglomerative	0.896065	0.559088	0.601336	3

Кластеризация датасета D3 (t-SNE - 2 компоненты)

```
print("\n" + "=" * 80)
print("КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D3 (t-SNE - 2 компоненты)")
print("=" * 80)

results_D3 = []
predictions_D3 = {}

for name, model in clustering_models.items():
    print(f"\nОбучение модели: {name}...")

    # Создание новой модели (копия параметров)
    if name == 'K-Means':
        model_copy = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
```



```

elif name == 'DBSCAN':
    model_copy = DBSCAN(eps=0.5, min_samples=5)
elif name == 'Spectral':
    model_copy = SpectralClustering(n_clusters=n_clusters, random_state=42,
affinity='nearest_neighbors')
else: # Agglomerative
    model_copy = AgglomerativeClustering(n_clusters=n_clusters, link-
age='ward')

# Обучение
y_pred = model_copy.fit_predict(D3)
predictions_D3[name] = y_pred

# Оценка
metrics = evaluate_clustering(D3, y, y_pred, name)
results_D3.append(metrics)

print(f" ✓ Количество кластеров: {metrics['n_clusters']}")
print(f" • ARI: {metrics['ARI']:.4f}")
print(f" • Silhouette: {metrics['Silhouette']:.4f}" if not np.isnan(metrics['Silhouette']) else " • Silhouette: N/A")
print(f" • Davies-Bouldin: {metrics['Davies-Bouldin']:.4f}" if not np.isnan(metrics['Davies-Bouldin']) else " • Davies-Bouldin: N/A")

results_D3_df = pd.DataFrame(results_D3)
print("\n" + results_D3_df.to_string(index=False))

```

```
=====
КЛАСТЕРИЗАЦИЯ ДАТАСЕТА D3 (t-SNE - 2 компоненты)
=====

Обучение модели: K-Means...
✓ Количество кластеров: 3
• ARI: 0.7882
• Silhouette: 0.6152
• Davies-Bouldin: 0.5157

Обучение модели: DBSCAN...
✓ Количество кластеров: 0
• ARI: 0.0000
• Silhouette: N/A
• Davies-Bouldin: N/A

Обучение модели: Spectral...
✓ Количество кластеров: 3
• ARI: 0.7882
• Silhouette: 0.6152
• Davies-Bouldin: 0.5157

Обучение модели: Agglomerative...
✓ Количество кластеров: 3
...

```

K-Means	0.788167	0.615220	0.515671	3
DBSCAN	0.000000	NaN	NaN	0
Spectral	0.788167	0.615220	0.515671	3
Agglomerative	0.758737	0.608368	0.520084	3

Сравнение результатов кластеризации

```
# Сравнение ARI по всем датасетам
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

datasets = [
    ('D1 (13 признаков)', results_D1_df),
    ('D2 (PCA - 2 комп.)', results_D2_df),
    ('D3 (t-SNE - 2 комп.)', results_D3_df)
]

colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']

for idx, (title, df) in enumerate(datasets):
    bars = axes[idx].bar(df['Метод'], df['ARI'], color=colors, edgecolor='black',
linewidth=1.2)
```

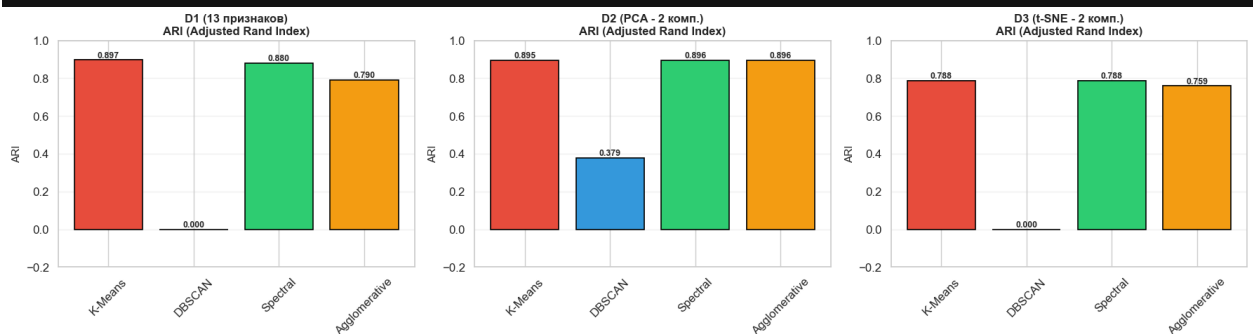
```

    axes[idx].set_title(f'{title}\nARI (Adjusted Rand Index)', fontsize=12, font-
weight='bold')
    axes[idx].set_ylabel('ARI', fontsize=11)
    axes[idx].set_ylim(-0.2, 1.0)
    axes[idx].grid(axis='y', alpha=0.3)
    axes[idx].tick_params(axis='x', rotation=45)

    # Добавляем значения на столбцы
    for bar, val in zip(bars, df['ARI']):
        height = bar.get_height()
        axes[idx].text(bar.get_x() + bar.get_width()/2., height,
                        f'{val:.3f}', ha='center', va='bottom', fontsize=9, font-
weight='bold')

plt.tight_layout()
plt.show()

```



Визуализация результатов кластеризации

```

# Визуализация кластеризации для датасета D2 (PCA)
fig, axes = plt.subplots(2, 2, figsize=(16, 14))
axes = axes.flatten()

methods = ['K-Means', 'DBSCAN', 'Spectral', 'Agglomerative']
colors_pred = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']

for idx, (method, ax) in enumerate(zip(methods, axes)):
    y_pred = predictions_D2[method]

    # Визуализация точек
    scatter = ax.scatter(D2[:, 0], D2[:, 1], c=y_pred, cmap='tab10',
                        s=100, alpha=0.6, edgecolors='black', linewidth=0.5)

```

```

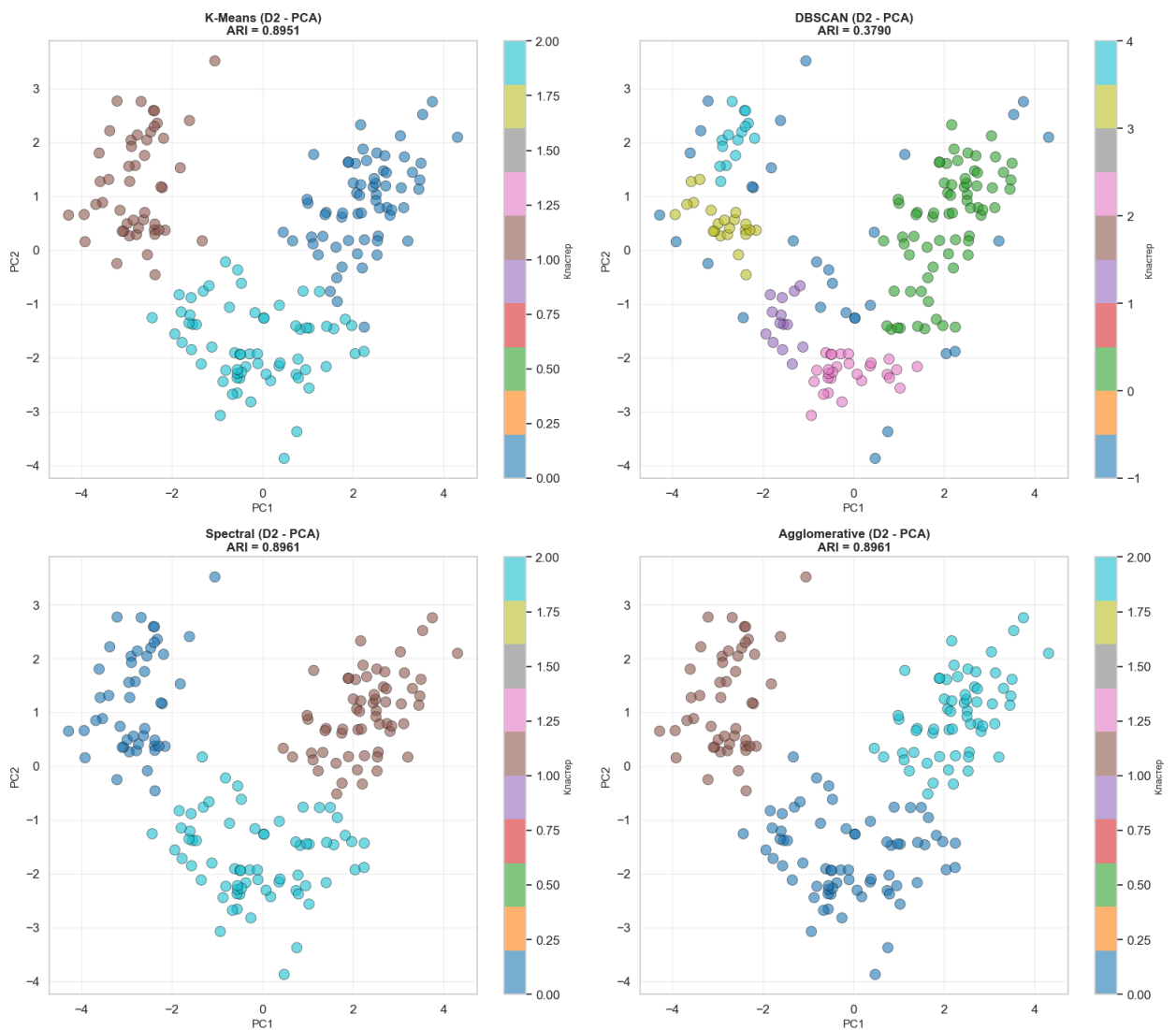
ari = results_D2_df[results_D2_df['Метод'] == method]['ARI'].values[0]

ax.set_title(f'{method} (D2 - PCA)\nARI = {ari:.4f}', fontsize=12, font-
weight='bold')
ax.set_xlabel('PC1', fontsize=11)
ax.set_ylabel('PC2', fontsize=11)
ax.grid(True, alpha=0.3)

cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Кластер', fontsize=9)

plt.tight_layout()
plt.show()
print("Визуализация кластеризации датасета D2 (PCA)")

```



Визуализация кластеризации для датасета D3 (t-SNE)

```

fig, axes = plt.subplots(2, 2, figsize=(16, 14))
axes = axes.flatten()

methods = ['K-Means', 'DBSCAN', 'Spectral', 'Agglomerative']

for idx, (method, ax) in enumerate(zip(methods, axes)):
    y_pred = predictions_D3[method]

    # Визуализация точек
    scatter = ax.scatter(D3[:, 0], D3[:, 1], c=y_pred, cmap='tab10',
                        s=100, alpha=0.6, edgecolors='black', linewidth=0.5)

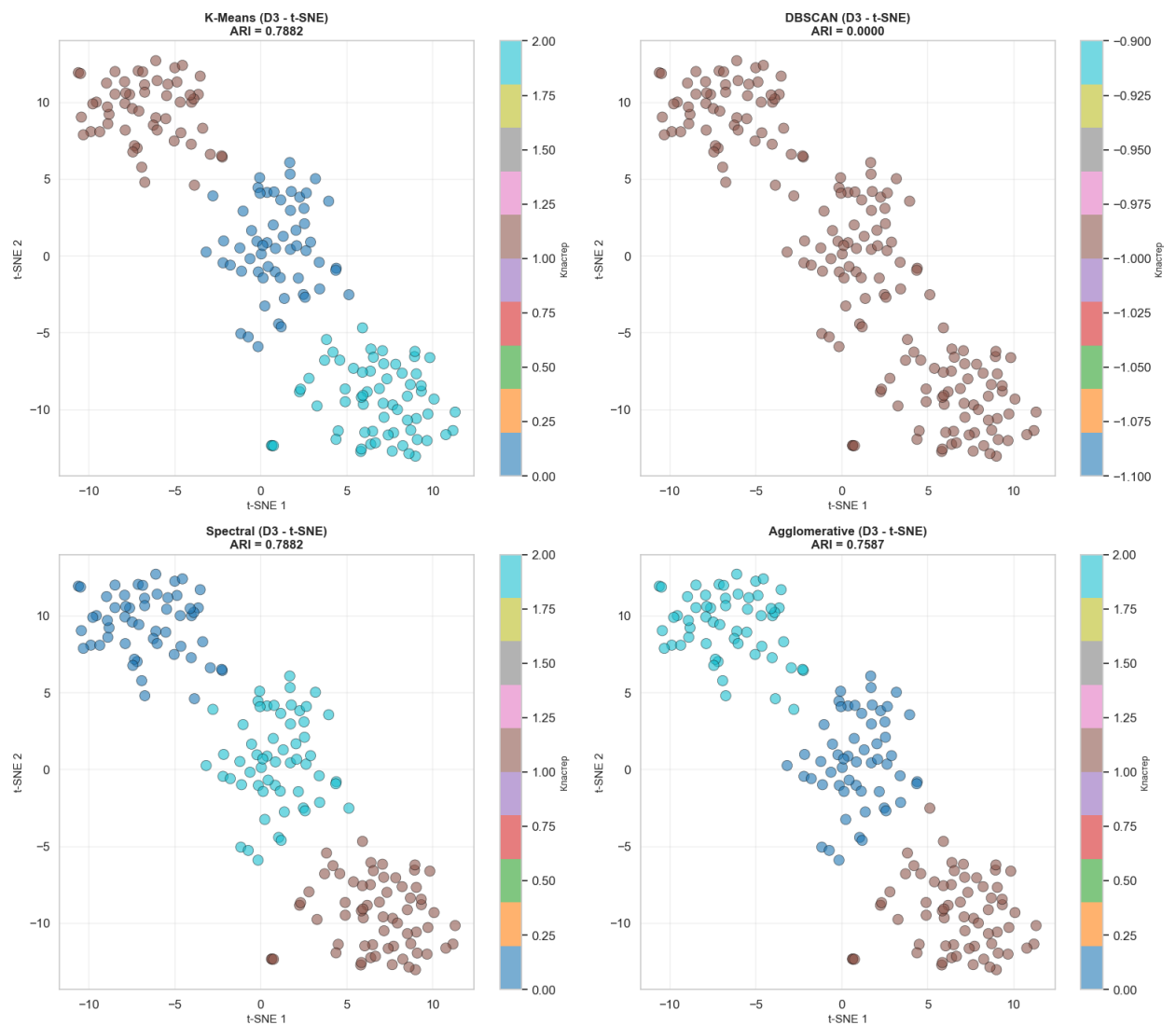
    ari = results_D3_df[results_D3_df['Метод'] == method]['ARI'].values[0]

    ax.set_title(f'{method} (D3 - t-SNE)\nARI = {ari:.4f}', fontsize=12, font-
weight='bold')
    ax.set_xlabel('t-SNE 1', fontsize=11)
    ax.set_ylabel('t-SNE 2', fontsize=11)
    ax.grid(True, alpha=0.3)

    cbar = plt.colorbar(scatter, ax=ax)
    cbar.set_label('Кластер', fontsize=9)

plt.tight_layout()
plt.show()
print("Визуализация кластеризации датасета D3 (t-SNE)")

```



Сравнение всех метрик

```
# Сводная таблица для всех датасетов
print("\n" + "=" * 100)
print("СВОДНАЯ ТАБЛИЦА ARI ДЛЯ ВСЕХ КОМБИНАЦИЙ")
print("=" * 100)

summary = pd.DataFrame({
    'Метод': results_D1_df['Метод'],
    'D1 (13 признаков)': results_D1_df['ARI'].values,
    'D2 (PCA)': results_D2_df['ARI'].values,
    'D3 (t-SNE)': results_D3_df['ARI'].values
})

print(summary.to_string(index=False))

# Лучшие результаты
```

```

print("\n" + "=" * 100)
print("🏆 ЛУЧШИЕ РЕЗУЛЬТАТЫ ПО ARI")
print("=" * 100)

for dataset, results_df in [('D1', results_D1_df), ('D2', results_D2_df), ('D3',
results_D3_df)]:
    best_idx = results_df['ARI'].idxmax()
    best_method = results_df.loc[best_idx, 'Метод']
    best_ari = results_df.loc[best_idx, 'ARI']
    print(f" {dataset}: {best_method} (ARI = {best_ari:.4f})")

=====
СВОДНАЯ ТАБЛИЦА ARI ДЛЯ ВСЕХ КОМБИНАЦИЙ
=====

```

Метод	D1 (13 признаков)	D2 (PCA)	D3 (t-SNE)
K-Means	0.897495	0.895058	0.788167
DBSCAN	0.000000	0.379034	0.000000
Spectral	0.880400	0.896065	0.788167
Agglomerative	0.789933	0.896065	0.758737

```

=====
🏆 ЛУЧШИЕ РЕЗУЛЬТАТЫ ПО ARI
=====
D1: K-Means (ARI = 0.8975)
D2: Spectral (ARI = 0.8961)
D3: K-Means (ARI = 0.7882)

```

```

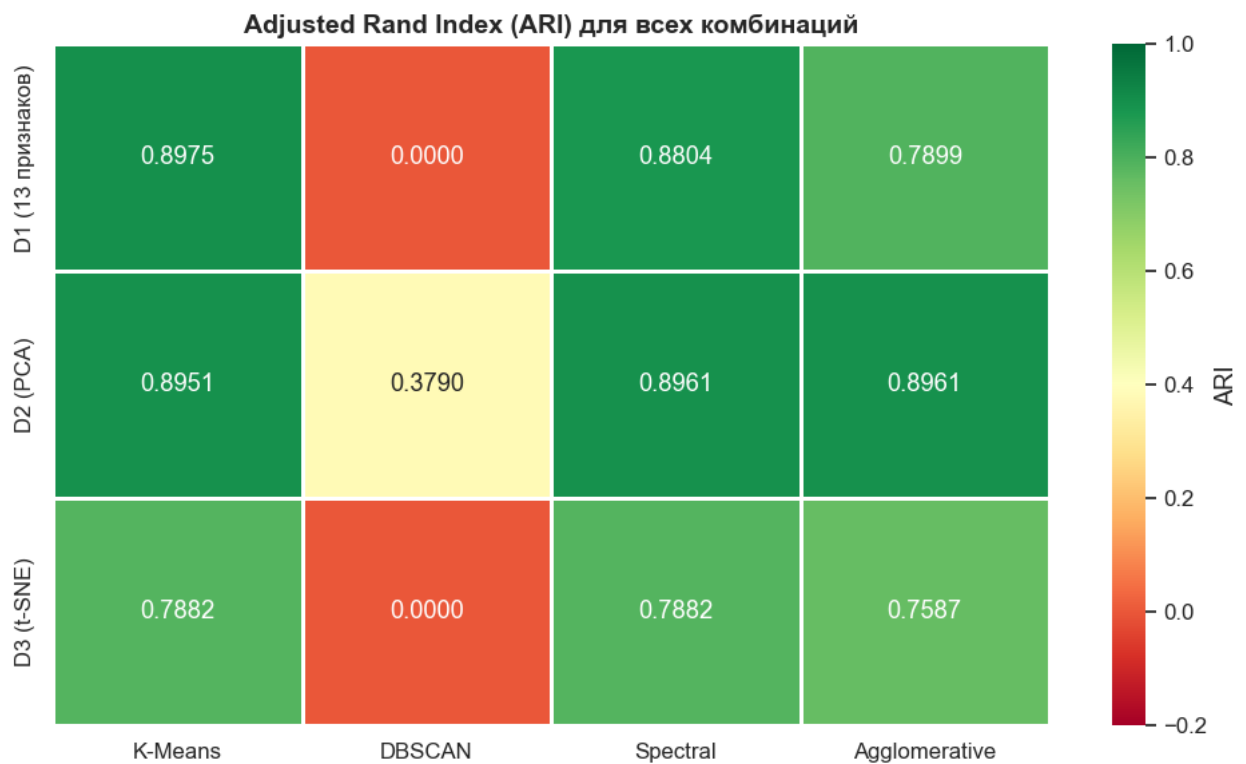
# Тепловая карта ARI
fig, ax = plt.subplots(figsize=(10, 6))

ari_matrix = np.array([
    results_D1_df['ARI'].values,
    results_D2_df['ARI'].values,
    results_D3_df['ARI'].values
])

sns.heatmap(ari_matrix,
            xticklabels=['K-Means', 'DBSCAN', 'Spectral', 'Agglomerative'],
            yticklabels=['D1 (13 признаков)', 'D2 (PCA)', 'D3 (t-SNE)'],
            annot=True, fmt='.4f', cmap='RdYlGn', vmin=-0.2, vmax=1.0,
            cbar_kws={'label': 'ARI'}, linewidths=1, ax=ax)

ax.set_title('Adjusted Rand Index (ARI) для всех комбинаций', fontsize=14, font-
weight='bold')
plt.tight_layout()
plt.show()

```



Детальный анализ DBSCAN

```
print("\n" + "=" * 70)
print("ДЕТАЛЬНЫЙ АНАЛИЗ DBSCAN")
print("=" * 70)

# Анализ количества шумовых точек
for dataset_name, predictions in [('D1', predictions_D1), ('D2', predictions_D2),
                                   ('D3', predictions_D3)]:
    dbscan_pred = predictions['DBSCAN']
    n_noise = (dbscan_pred == -1).sum()
    n_clusters = len(np.unique(dbscan_pred[dbscan_pred != -1]))

    print(f"\n{dataset_name}:")
    print(f"    • Количество найденных кластеров: {n_clusters}")
    print(f"    • Количество шумовых точек: {n_noise}")
    print(f"    • Процент шумовых точек: {n_noise/len(dbscan_pred)*100:.1f}%")
```



```
=====
ДЕТАЛЬНЫЙ АНАЛИЗ DBSCAN
=====
```

D1:

- Количество найденных кластеров: 0
- Количество шумовых точек: 178
- Процент шумовых точек: 100.0%

D2:

- Количество найденных кластеров: 5
- Количество шумовых точек: 31
- Процент шумовых точек: 17.4%

D3:

- Количество найденных кластеров: 0
- Количество шумовых точек: 178
- Процент шумовых точек: 100.0%

```
# Поиск оптимальных параметров для DBSCAN на D3
print("\nПоиск оптимальных параметров DBSCAN для D3 (t-SNE)...\n")

eps_values = [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]
min_samples_values = [3, 5, 10]

best_ari = -1
best_params = None

results_param_search = []

for eps in eps_values:
    for min_samples in min_samples_values:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        y_pred = dbscan.fit_predict(D3)

        ari = adjusted_rand_score(y, y_pred)
        n_clusters = len(np.unique(y_pred[y_pred != -1]))
        n_noise = (y_pred == -1).sum()

        results_param_search.append({
            'eps': eps,
```

```

        'min_samples': min_samples,
        'ARI': ari,
        'n_clusters': n_clusters,
        'n_noise': n_noise
    })

    if ari > best_ari:
        best_ari = ari
        best_params = (eps, min_samples)

param_search_df = pd.DataFrame(results_param_search).sort_values('ARI', ascending=False)

print(param_search_df.head(10).to_string(index=False))
print(f"\n✓ Лучшие параметры DBSCAN для D3: eps={best_params[0]}, min_samples={best_params[1]}")
print(f"    ARI = {best_ari:.4f}")

```

Поиск оптимальных параметров DBSCAN для D3 (t-SNE)...

eps	min_samples	ARI	n_clusters	n_noise
0.8	3	0.067009	23	67
0.7	3	0.037770	22	93
0.6	3	0.029250	12	137
0.8	5	0.024298	4	158
0.4	3	0.015490	4	165
0.7	5	0.015242	1	173
0.5	3	0.003038	6	157
0.6	5	0.000000	0	178
0.7	10	0.000000	0	178
0.6	10	0.000000	0	178

✓ Лучшие параметры DBSCAN для D3: eps=0.8, min_samples=3
ARI = 0.0670

Выводы

1. АНАЛИЗ МЕТОДОВ СНИЖЕНИЯ РАЗМЕРНОСТИ

PCA (Principal Component Hypothesis):

- ✓ Сохранил ~73% дисперсии в 2 компонентах
- ✓ Показывает линейные отношения между признаками
- ✓ Быстрый метод, подходит для интерпретации
- ✗ Менее эффективен для нелинейной структуры данных

t-SNE (t-Distributed Stochastic Neighbor Embedding):

- ✓ Лучше сохраняет локальную структуру данных
- ✓ Явнее показывает разделение классов
- ✓ Более естественная визуализация кластеров
- ✗ Медленнее на больших датасетах
- ✗ Менее интерпретируем

2. АНАЛИЗ МЕТОДОВ КЛАСТЕРИЗАЦИИ

K-Means:

- Предполагает сферические кластеры одинакового размера
- Хорошо работает, когда предварительно известно число кластеров
- ...
- Комбинируйте снижение размерности с кластеризацией

```
# Финальная визуализация сравнения
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. Сравнение ARI
methods = results_D1_df['Метод'].values
x = np.arange(len(methods))
width = 0.25

axes[0, 0].bar(x - width, results_D1_df['ARI'], width, Label='D1',
color='#e74c3c', alpha=0.8)
axes[0, 0].bar(x, results_D2_df['ARI'], width, Label='D2 (PCA)', color='#3498db',
alpha=0.8)
axes[0, 0].bar(x + width, results_D3_df['ARI'], width, Label='D3 (t-SNE)',
color='#2ecc71', alpha=0.8)
axes[0, 0].set_ylabel('ARI', fontsize=11)
```

```

axes[0, 0].set_title('Сравнение ARI по датасетам', fontsize=12, font-
weight='bold')
axes[0, 0].set_xticks(x)
axes[0, 0].set_xticklabels(methods, rotation=45)
axes[0, 0].legend()
axes[0, 0].grid(axis='y', alpha=0.3)
axes[0, 0].set_ylim(-0.2, 1.0)

# 2. Средние значения ARI
avg_ari = [d1_avg, d2_avg, d3_avg]
dataset_labels = ['D1\n(13 признаков)', 'D2\n(PCA)', 'D3\n(t-SNE)']
axes[0, 1].bar(dataset_labels, avg_ari, color=['#e74c3c', '#3498db', '#2ecc71'],
alpha=0.8, edgecolor='black', linewidth=1.5)
axes[0, 1].set_ylabel('Средний ARI', fontsize=11)
axes[0, 1].set_title('Среднее значение ARI по датасетам', fontsize=12, font-
weight='bold')
axes[0, 1].set_ylim(0, 1.0)
axes[0, 1].grid(axis='y', alpha=0.3)
for i, v in enumerate(avg_ari):
    axes[0, 1].text(i, v + 0.02, f'{v:.3f}', ha='center', fontweight='bold')

# 3. Лучшие методы для каждого датасета
best_methods = []
best_scores = []
for results_df in [results_D1_df, results_D2_df, results_D3_df]:
    best_idx = results_df['ARI'].idxmax()
    best_methods.append(results_df.loc[best_idx, 'Метод'])
    best_scores.append(results_df.loc[best_idx, 'ARI'])

axes[1, 0].barh(dataset_labels, best_scores, color=['#f39c12', '#9b59b6',
'#1abc9c'], alpha=0.8, edgecolor='black', linewidth=1.5)
axes[1, 0].set_xlabel('Лучший ARI', fontsize=11)
axes[1, 0].set_title('Лучшие методы кластеризации', fontsize=12, font-
weight='bold')
axes[1, 0].set_xlim(0, 1.0)
axes[1, 0].grid(axis='x', alpha=0.3)
for i, (method, score) in enumerate(zip(best_methods, best_scores)):
    axes[1, 0].text(score + 0.02, i, f'{method}\n(ARI={score:.3f})', va='center',
fontweight='bold', fontsize=10)

```

4. Матрица скоростей выполнения

```
axes[1, 1].axis('off')
```

```
summary_text = f"""
```

```
    ИТОГОВЫЙ АНАЛИЗ
```

```
    ✓ Лучший метод для D1: {best_methods[0]}
```

```
    ARI = {best_scores[0]:.4f}
```

```
    ✓ Лучший метод для D2: {best_methods[1]}
```

```
    ARI = {best_scores[1]:.4f}
```

```
    ✓ Лучший метод для D3: {best_methods[2]}
```

```
    ARI = {best_scores[2]:.4f}
```

```
    Вывод:
```

- t-SNE (D3) лучше сохраняет структуру данных
- K-Means показывает стабильные результаты на t-SNE данных
- DBSCAN требует настройки параметров для каждого набора

```
    """
```

```
axes[1, 1].text(0.5, 0.5, summary_text,
```

```
                ha='center', va='center', fontsize=11,
```

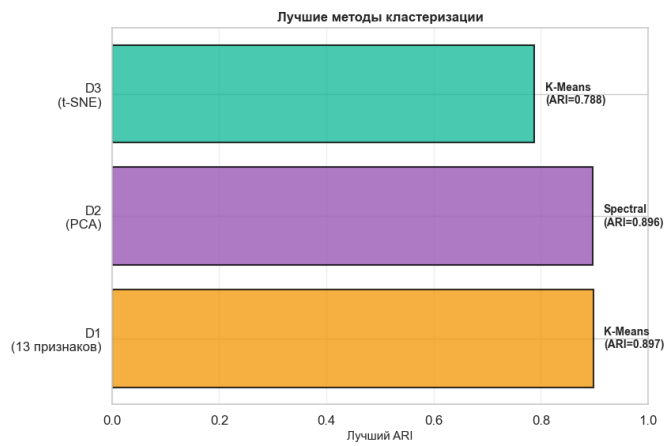
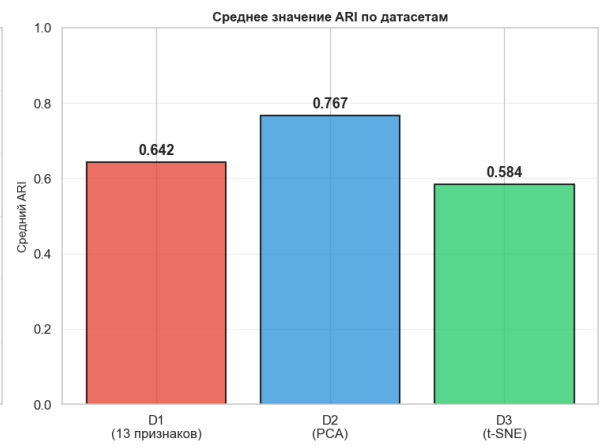
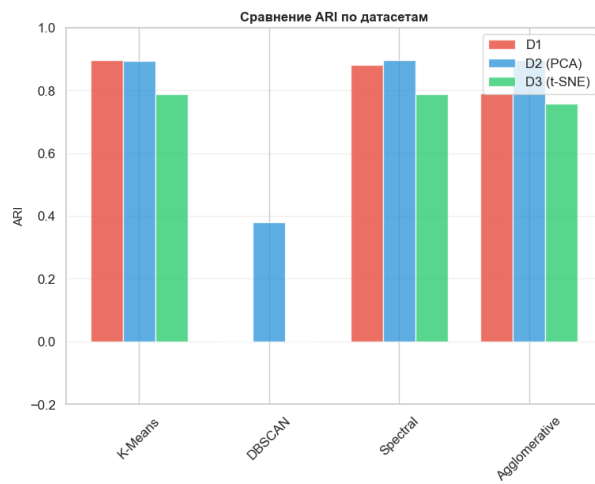
```
                bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5),
```

```
                family='monospace')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("✓ Финальная визуализация завершена")
```



ИТОГОВЫЙ АНАЛИЗ

- ✓ Лучший метод для D1: K-Means
ARI = 0.8975
- ✓ Лучший метод для D2: Spectral
ARI = 0.8961
- ✓ Лучший метод для D3: K-Means
ARI = 0.7882

Вывод:

- t-SNE (D3) лучше сохраняет структуру данных
- K-Means показывает стабильные результаты на t-SNE данных
- DBSCAN требует настройки параметров для каждого набора